



***Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado***

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

**Tema 59. Conceptos y fundamentos de sistemas operativos.
Evolución y tendencias**

AUTOR: Juan Guevara Escudero

Actualización 2015

ÍNDICE

1	INTRODUCCIÓN	4
2	COMPONENTES DEL SISTEMA OPERATIVO.....	6
2.1	ADMINISTRACIÓN DE PROCESOS	7
2.1.1	EL CONCEPTO DE PROCESO	7
2.1.2	SUBPROCESOS	8
2.1.3	PLANIFICACIÓN DEL PROCESADOR	9
2.1.4	PROBLEMAS DE CONCURRENCIA.....	10
2.2	ADMINISTRACIÓN DE MEMORIA	12
2.2.1	MANEJO DE MEMORIA EN SISTEMAS MONOUSUARIO SIN INTERCAMBIO.....	12
2.2.2	MULTIPROGRAMACIÓN EN MEMORIA REAL	13
2.3	ADMINISTRACIÓN DE ENTRADA / SALIDA	20
2.3.1	DISPOSITIVOS DE ENTRADA/SALIDA.....	20
2.3.2	CONTROLADORES DE DISPOSITIVOS (TERMINALES Y DISCOS DUROS)	20
2.3.3	ACCESO DIRECTO A MEMORIA (DMA)	21
2.3.4	PRINCIPIOS DEL SOFTWARE DE ENTRADA/SALIDA.....	21
2.3.5	RELOJES	22
2.3.6	PLUG AND PLAY	22
2.3.7	LA INTERFAZ DE USUARIO	23
2.4	SISTEMAS DE ARCHIVOS.....	24
2.4.1	ALMACENAMIENTO FÍSICO DE DATOS.....	24
2.4.2	ASIGNACIÓN DEL ESPACIO DE ALMACENAMIENTO.....	25
2.4.3	MÉTODOS DE ACCESO EN LOS SISTEMAS DE ARCHIVOS	26
2.4.4	OPERACIONES SOPORTADAS POR EL SUBSISTEMA DE ARCHIVOS	26
2.5	CONCLUSIÓN	27
3	CLASIFICACIÓN DE LOS SISTEMAS OPERATIVOS	28
3.1	SISTEMAS OPERATIVOS POR SU CAPACIDAD.....	29
3.2	SISTEMAS OPERATIVOS POR SU ESTRUCTURA	30
3.2.1	ESTRUCTURA MONOLÍTICA.....	30
3.2.2	ESTRUCTURA JERÁRQUICA O EN CAPAS	31
3.2.3	MÁQUINA VIRTUAL	32
3.2.4	CLIENTE-SERVIDOR (MICROKERNEL)	32
3.2.5	HÍBRIDO	33
3.3	SISTEMAS OPERATIVOS POR LA FORMA DE OFRECER SUS SERVICIOS.	33
3.3.1	SISTEMAS OPERATIVOS DE RED.....	33
3.3.2	SISTEMAS OPERATIVOS DISTRIBUIDOS.....	33
3.4	SISTEMAS OPERATIVOS POR EL ENTORNO HARDWARE EN EL QUE SE EJECUTAN	35
3.4.1	SISTEMAS OPERATIVOS DE MAINFRAME.....	35
3.4.2	SISTEMAS OPERATIVOS DE SERVIDOR	35
3.4.3	SISTEMAS OPERATIVOS MULTIPROCESADOR	35
3.4.4	SISTEMAS OPERATIVOS DE PC	35
3.4.5	SISTEMAS OPERATIVOS DE TIEMPO REAL	36
3.4.6	SISTEMAS OPERATIVOS INTEGRADOS.....	36
3.4.7	SISTEMAS OPERATIVOS MÓVILES	36
3.4.8	SISTEMAS OPERATIVOS DE TARJETA INTELIGENTE	36

3.5	CONCLUSIÓN	36
4	EVOLUCIÓN Y TENDENCIAS DE LOS SISTEMAS OPERATIVOS.....	37
4.1	INTRODUCCIÓN.....	37
4.2	GENERACIONES DE LOS SISTEMAS OPERATIVOS	37
4.2.1	PRIMERA GENERACIÓN (1945-1955): TUBOS DE VACÍO Y TABLEROS ENCHUFABLES.	37
4.2.2	SEGUNDA GENERACIÓN (1955-1965): TRANSISTORES Y SISTEMAS DE LOTES.	37
4.2.3	TERCERA GENERACIÓN (1965-1980): CIRCUITOS INTEGRADOS (CI) Y MULTIPROGRAMACIÓN.	37
4.2.4	CUARTA GENERACIÓN (1980-HASTA HOY): ORDENADORES PERSONALES.....	38
4.3	TENDENCIAS DE LOS SISTEMAS OPERATIVOS EN LA ACTUALIDAD	38
4.3.1	SISTEMAS DE 32BITS Y DE 64 BITS	38
4.3.2	SISTEMAS OPERATIVOS MÓVILES	39
4.4	CONCLUSIÓN	43

1 Introducción

El Sistema Operativo es el software más importante de un ordenador, ya que es el que controla todos los recursos físicos del ordenador y el encargado de ofrecer las bases y servicios sobre los que se ejecutarán el resto de aplicaciones software. El Sistema Operativo es el software más cercano al propio hardware del ordenador y sobre él se desarrollan las aplicaciones, cuyo objetivo principal es el de resolver problemas de los usuarios.

Los Sistemas Operativos responden a dos funcionalidades principales, en función del punto de vista:

- **Desde el punto de vista del usuario**, el sistema operativo tiene como función presentar una máquina virtual o máquina ampliada más fácil de programar y manejar que el hardware asociado. Permite la Creación de programas (edición, compilación, enlazado de librerías), su Ejecución (carga, inicialización de dispositivos de E/S, inicialización de ficheros), Acceso a los dispositivos de E/S (instrucciones propias y señales de control de cada dispositivo a través de drivers), Acceso al sistema de ficheros (naturaleza del dispositivo, formato de ficheros, protección frente a múltiples usuarios), Acceso al sistema (sistemas compartidos, gestión de acceso de cada usuario).
- **Desde el punto de vista de la máquina**, el sistema se presenta como un administrador de recursos cuya función es controlar todos los elementos que forman un ordenador. Resolviendo los conflictos que se generan por el uso concurrente de los recursos únicos del sistema entre los diferentes programas y usuarios.

Un ordenador moderno consta de multitud de componentes, entre los que destacan: uno o más procesadores, memoria, relojes, discos, interfaces de redes, etc. En conjunto se trata de un sistema complejo y el Sistema Operativo es la capa que abstrae a los desarrolladores de aplicaciones de la complejidad de comunicarse directamente con esos componentes hardware. Si los desarrolladores tuvieran que desarrollar su aplicación teniendo en cuenta los componentes Hardware con los que quieren que se comunique su aplicación, dichos desarrollos serían inviables y muy costosos.

A continuación se muestra una figura muy representativa de las diferentes capas Software existentes entre el Hardware de la máquina y las aplicaciones finales:



Figura 1: Las diferentes capas Software existentes entre el Hardware de la máquina y las aplicaciones finales

Aunque gran parte del software del sistema es proporcionado por el fabricante del Sistema Operativo, no se considera parte del mismo. El sistema operativo es la parte del software que se ejecuta en modo kernel o modo supervisor.

Todos los Sistemas Operativos incorporan ciertos **conceptos básicos**, que son fundamentales para entenderlos. A modo de introducción, citaremos los más importantes:

- **Procesos:** el proceso es básicamente, un programa en ejecución. Cada proceso lleva asociada la información referida a su estado y recursos asignados, necesaria para su gestión por parte del sistema operativo.
- **Administración de memoria:** todo ordenador tiene una memoria principal que contiene los programas en ejecución. En los sistemas más simples solamente hay un programa a la vez en la memoria, y para ejecutar un segundo programa es preciso desalojar el primero. Los Sistemas Operativos más avanzados permiten que haya varios programas en la memoria al mismo tiempo, precisando de algún mecanismo de control y administración que gestione adecuadamente su correcta utilización.
- **Entrada/Salida:** todo ordenador cuenta con dispositivos físicos para obtener entradas y producir salidas. Existen muchos tipos de dispositivos diferentes, como teclados, monitores, impresoras, discos, etc., que deben ser administrados por el sistema operativo, labor que se lleva a cabo en el subsistema de E/S.
- **Archivos:** a fin de contar con un medio de guardar la información, casi todos los Sistemas Operativos manejan el concepto de sistema de archivos, como una organización del almacén de la información, y el concepto de archivo como la unidad básica de almacenamiento.
- **Seguridad:** los ordenadores contienen grandes cantidades de información que los usuarios a menudo consideran confidencial. Corresponde al sistema operativo controlar la seguridad del sistema para que los diferentes usuarios y procesos puedan acceder únicamente a la información para la que están autorizados.

2 Componentes del Sistema Operativo

Un Sistema Operativo tiene cuatro cometidos principales:

- La gestión de procesos.
- La gestión de memoria.
- El sistema de archivos.
- La gestión de la Entrada/Salida.

Los componentes encargados de llevar a cabo estos cometidos dentro del Sistema Operativo son:

- **Administrador de procesos:** encargado de crear y eliminar procesos, de suspender y reanudar su ejecución y de proporcionar mecanismos de sincronización, de comunicación de procesos y de manejo de bloqueos.
- **Administrador de memoria principal:** controla las zonas de memoria utilizadas y quién las utiliza. Decide qué procesos se cargarán en memoria si queda espacio disponible. Asigna y recupera espacio no necesitado.
- **Administrador del sistema de E/S:** encargado de gestionar los dispositivos de hardware específico, y de presentar una interfaz general común para interactuar con los diferentes manejadores.
- **Administrador de archivos:** es el encargado de la gestión del espacio de almacenamiento en disco. Se ocupa de los procesos relacionados con la creación y borrado de archivos, el establecimiento de la correspondencia entre archivos y almacenamiento secundario, la gestión de directorios y la gestión de copias de seguridad.
- **Sistema de protección:** encargado de controlar el acceso a los recursos, garantiza el acceso con la debida autorización y evita accesos no permitidos.
- **Sistema de comunicación:** encargado de gestionar los accesos a la red, facilita la conexión a través de la red de los distintos procesadores (en sistemas multiordenador). Facilita el acceso a los recursos compartidos.

2.1 Administración de procesos

2.1.1 El concepto de proceso

Un proceso no es más que un programa en ejecución que necesita utilizar recursos del ordenador para realizar su tarea: tiempo de CPU, memoria, archivos y dispositivos de E/S.

El Sistema Operativo es el responsable de:

- Crear y destruir procesos.
- Parar y reanudar procesos.
- Ofrecer mecanismos para que los procesos puedan comunicarse y se sincronicen.

Los ordenadores modernos son multitarea, es decir, pueden hacer varias cosas al mismo tiempo. Los recursos del ordenador son únicos, pero la CPU cambia de un programa a otro, ejecutando cada uno durante unas decenas o centenas de milisegundos, provocando al usuario la ilusión de paralelismo.

La diferencia entre un proceso y un programa es sutil, pero crucial. El programa es el conjunto de instrucciones que indican cómo realizar algún cometido; un proceso es una actividad de algún tipo: tiene un programa, entradas, salidas y un estado. Varios procesos pueden compartir el mismo programa, también pueden compartir un único procesador, empleándose algún algoritmo de planificación para determinar cuándo hay que dejar de trabajar en un proceso y atender otro.

Para implementar el modelo de procesos, el Sistema Operativo mantiene una tabla llamada tabla de procesos, con una entrada por proceso. Esta entrada contiene información acerca del estado del proceso y otra información necesaria para restituir el proceso en el punto donde se interrumpió y proseguir su ejecución.

Hay cuatro sucesos principales que provocan la **creación de procesos**:

- **Inicialización del sistema:** el arranque de un sistema operativo conlleva la creación de varios procesos, algunos son interactivos o en primer plano, otros son procesos en segundo plano que no están asociados con un usuario en particular y desarrollan alguna función específica.
- **Ejecución de una llamada al sistema por parte de un proceso en ejecución:** el sistema operativo proporciona llamadas para la creación de procesos que pueden ser invocadas por otros procesos.
- **Solicitud de un usuario para crear un proceso:** en los sistemas interactivos, los usuarios pueden iniciar un programa, tecleando un comando o pulsando un icono. Ambas acciones inician un nuevo proceso donde se ejecuta el programa seleccionado.
- **Inicio de un trabajo por lotes:** aplicable a los mainframes con sistemas por lotes. Los usuarios envían una lista de trabajos y el sistema operativo crea los procesos necesarios para ejecutarlos adecuadamente.

Los procesos terminan en algún momento, debido a una de las siguientes condiciones:

1. Terminación **normal** (voluntaria).
2. Terminación por **error** (voluntaria).
3. Error **fatal** (involuntaria).
4. Terminado por **otro proceso** (involuntaria).

La mayoría de procesos termina porque ya realizó su trabajo, o porque el usuario les pide que finalicen (caso de procesos interactivos). Puede que el proceso descubra que se ha producido un error y decida terminar la ejecución por su cuenta, o puede que se produzca un error que el proceso no controle y, en ese caso, es el sistema operativo el que decide su cancelación. Por último, contando

con los permisos adecuados, el sistema operativo cuenta con mecanismos para permitir a un proceso solicitar y obtener la cancelación de otros procesos en ejecución.

Hay veces que un proceso dispone de tiempo de CPU para su ejecución, pero no puede continuar porque depende de alguna circunstancia externa que se lo impide, como recibir datos desde otro proceso o desde un dispositivo externo al sistema. Esta situación es distinta del hecho de un proceso que no se ejecuta porque la CPU está ocupada en otro diferente. La Gestión de procesos asigna a cada proceso del sistema un estado que puede ser:

- **En ejecución:** el proceso está usando la CPU en ese instante
- **Listo:** el proceso puede ejecutarse, pero está detenido de forma temporal para que pueda ejecutarse otro proceso.
- **Bloqueado:** el proceso no puede ejecutarse mientras no ocurra cierto suceso externo a él.

Cuatro son las posibles transiciones que pueden darse entre los estados de un proceso:

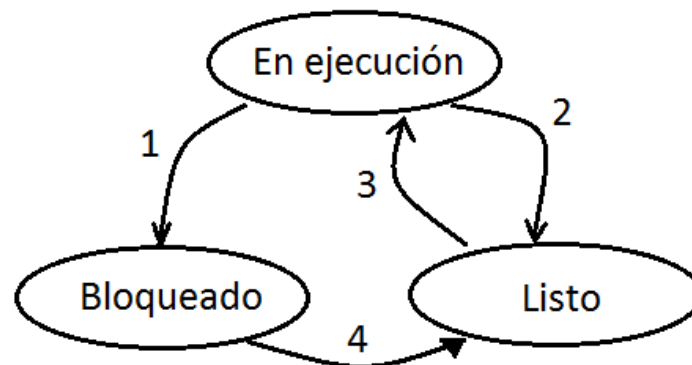


Figura 2: Transiciones entre los estados de un proceso

1. Un proceso se bloquea, por ejemplo, porque está esperando una entrada.
2. El planificador escoge otro proceso.
3. El planificador decide ejecutar este proceso.
4. Cuando las entradas ya han llegado, el proceso está listo para ejecutarse.

2.1.2 Subprocesos

En los Sistemas Operativos, el proceso tiene asignado una serie de recursos del sistema, conocido como entorno. El proceso se puede dividir en subprocessos que utilicen ese entorno asignado al proceso de forma paralela. Los subprocessos comparten el mismo espacio de direcciones, archivos abiertos y otros recursos del proceso, recibiendo a veces el nombre de procesos ligeros. No existe protección entre subprocessos.

Un subprocesso puede estar en cualquiera de los cuatro estados de ejecución de un proceso (En ejecución, Listo, Bloqueado y Terminado) y puede realizar llamadas al sistema para crear nuevos subprocessos.

La utilidad de los subprocessos se justifica cuando en diversas aplicaciones se están realizando varias actividades al mismo tiempo, por lo que algunas de ellas podrían bloquearse de vez en cuando. Descomponer la aplicación en múltiples subprocessos que se ejecutan casi en paralelo simplifica el modelo de programación.

Un segundo argumento a favor de los subprocessos es que, al no estar enlazados con recursos, son más fáciles de crear y destruir que los procesos (a veces del orden de 100 veces más rápidos).

Un tercer motivo para tener subprocessos se relaciona con el rendimiento. Los subprocessos no mejoran el rendimiento cuando todos hacen un uso intensivo de CPU, pero la mejora es notable

cuando el trabajo se reparte equilibradamente entre Proceso y E/S, y aún lo es más en sistemas con múltiples CPUs en los que es posible un verdadero paralelismo.

2.1.3 Planificación del procesador

La planificación del procesador se refiere al momento y el tiempo que se decide utilizar por parte de los procesos del sistema. Si el sistema es monousuario y monotarea está claro, pero en el resto de los sistemas esto es crucial para el buen funcionamiento del sistema.

- Niveles de planificación

En los sistemas de planificación generalmente se identifican tres niveles: el alto, el medio y el bajo. El nivel alto (scheduler) decide qué trabajos (conjunto de procesos) son candidatos a convertirse en procesos compitiendo por los recursos del sistema; el nivel intermedio decide qué procesos se suspenden o reanudan para lograr ciertas metas de rendimiento, mientras que el planificador de bajo nivel (dispatcher) es el que decide qué proceso, de los que ya están listos (y que en algún momento pasó por los otros dos planificadores) es al que corresponde estar ejecutándose en la unidad central de proceso. En este documento se revisarán principalmente los planificadores de bajo nivel porque son los que finalmente eligen el proceso en ejecución.

- Objetivos de la planificación

Una estrategia de planificación debe buscar que los procesos obtengan sus turnos de ejecución apropiadamente, junto con un buen rendimiento y minimización de la sobrecarga (overhead) del planificador mismo. En general, se buscan cinco objetivos principales:

- **Justicia o Imparcialidad:** todos los procesos son tratados de la misma forma, y en algún momento obtienen su turno de ejecución o intervalos de tiempo de ejecución hasta su terminación exitosa.
- **Maximizar la Producción (throughput):** el sistema debe de finalizar el mayor número de procesos por unidad de tiempo.
- **Minimizar el Tiempo de Respuesta (Turnaround):** cada usuario o proceso debe observar que el sistema les responde consistentemente a sus requerimientos.
- **Evitar el aplazamiento indefinido:** los procesos deben terminar en un plazo finito de tiempo.
- **El sistema debe ser predecible:** Ante cargas de trabajo ligeras el sistema debe responder rápido y con cargas pesadas debe ir degradándose paulatinamente. Otro punto de vista de esto es que, si se ejecuta el mismo proceso bajo cargas similares en todo el sistema, la respuesta en todos los casos debe ser similar.

- Tipos de planificación

Existen dos tipos de planificación del procesador:

- **La planificación no apropiativa** es aquella en la cual, una vez que a un proceso le toca su turno de ejecución ya no puede ser suspendido, no se le puede arrebatar la CPU. Este esquema puede ser peligroso, ya que, si el proceso contiene ciclos infinitos, el resto de los procesos pueden quedar aplazados indefinidamente.
- **La planificación apropiativa** es aquella en que existe un reloj que lanza interrupciones periódicas en las cuales el planificador toma el control y se decide si el mismo proceso seguirá ejecutándose o se le da su turno a otro proceso.

- **Asignación del turno de ejecución**

A continuación se describen los algoritmos utilizados para asignar el turno de ejecución:

- **Por prioridad:** los procesos de mayor prioridad se ejecutan primero. Si existen varios procesos con la misma prioridad entre ellos pueden ejecutarse por su orden de llegada o por *round robin*. La **ventaja** de este algoritmo es que es flexible en la ejecución de los procesos. Su **desventaja** es que puede provocar aplazamiento indefinido en los procesos de baja prioridad.
- **El trabajo más corto primero (SJF Shortest Job First):** Se ejecutan primero aquellos trabajos que necesitan menos tiempo.
- **El primero en llegar, primero en ejecutarse:** es muy simple, los procesos reciben su turno conforme lleguen.

La **ventaja** de este algoritmo es que es justo y no provoca aplazamiento indefinido. La **desventaja** es que no aprovecha ninguna característica de los procesos y puede no servir para un proceso de tiempo real.

- **Round Robin:** también llamado por turno, consiste en asignar a cada proceso un intervalo de tiempo de ejecución (llamado time-slice), y cada vez que se vence ese intervalo se le asigna su turno a otro proceso. Los procesos están ordenados en una cola circular. La **ventaja** de este algoritmo es su simplicidad. Además, es justo y no provoca aplazamiento indefinido.
- **El tiempo restante más corto:** es parecido al del trabajo más corto primero, pero aquí se está calculando en todo momento cuánto tiempo resta para terminar a todos los procesos, incluyendo los nuevos. Aquél más próximo a finalizar es escogido para ejecutarse.
- **La tasa de respuesta más alta:** este algoritmo concede el turno de ejecución al proceso que produzca el mayor valor de la siguiente fórmula:

Valor = (tiempo que ha esperado + tiempo total para terminar) / tiempo total para terminar.

Es decir, dinámicamente el valor se va modificando y mejora un poco las deficiencias del algoritmo del trabajo más corto primero.

- **Por política:** es una forma de asignar el turno de ejecución en la cual se establece algún reglamento específico que el planificador debe obedecer. Por ejemplo, una política podría ser que todos los procesos reciban el mismo tiempo de uso de UCP en cualquier momento.

2.1.4 Problemas de concurrencia.

En los sistemas de tiempo compartido (aquéllos con varios usuarios, procesos, tareas, trabajos, y que reparten el uso de UCP entre éstos) se presentan muchos problemas debido a que los procesos compiten por los recursos del sistema que son únicos. Existe una multitud de recursos cuyo acceso debe ser controlado para evitar los problemas de la concurrencia.

El sistema operativo debe ofrecer mecanismos para sincronizar la ejecución de procesos y que no existan este tipo de problemas.

Es importante estudiar los **problemas generales de concurrencia**:

- **Condiciones de carrera o competencia:** la condición de carrera (race condition) ocurre cuando dos o más procesos acceden a un recurso compartido sin control, de manera que el resultado combinado de este acceso depende del orden de llegada.

- **Postergación o aplazamiento indefinido:** se mencionó en el apartado anterior y consiste en el hecho de que uno o varios procesos nunca reciban el suficiente tiempo de ejecución para terminar su tarea. Por ejemplo, que un proceso ocupe un recurso y lo marque como ocupado y que termine sin marcarlo como desocupado. Si algún otro proceso pide ese recurso, lo verá ocupado y esperará indefinidamente a que se desocupe.
- **Condición de espera circular:** se da cuando dos o más procesos forman una cadena de espera que los involucra a todos. Por ejemplo, supongamos que el proceso A tiene asignado el recurso cinta y el proceso B tiene asignado el recurso disco. Si el proceso A solicita el recurso disco y el proceso B solicita el recurso cinta, se forma una espera circular entre ambos procesos que se puede evitar quitándole a la fuerza un recurso a cualquiera de los dos procesos.
- **Condición de no apropiación:** esta condición no resulta precisamente de la concurrencia, pero juega un papel importante en este ambiente. Especifica que, si un proceso tiene asignado un recurso, dicho recurso no puede arrebatarle por ningún motivo, y estará disponible para dicho proceso hasta que éste suelte por su propia voluntad.
- **Condición de espera ocupada:** consiste en que un proceso pide un recurso que ya está asignado a otro proceso, debiendo cumplir la condición de no apropiación. En ese caso, el proceso estará gastando el resto de su time-slice chequeando si el recurso fue liberado. Es decir, desperdicia su tiempo de ejecución en esperar. La solución más común a este problema viene dada porque el sistema operativo se dé cuenta de esta situación y mande el proceso a una cola de espera, otorgándole inmediatamente el turno de ejecución a otro proceso.
- **Condición de exclusión mutua:** cuando un proceso utiliza un recurso del sistema realiza una serie de operaciones sobre el recurso, y después lo deja de usar. A la sección de código que usa ese recurso se le llama región crítica.

La condición de exclusión mutua establece que solamente se permite a un único proceso estar dentro de la misma región crítica. Esto es, en cualquier momento solamente un proceso puede usar un recurso a la vez. Para lograr la exclusión mutua se ideó también el concepto de región crítica. Existen distintas técnicas para lograr entrar a la región crítica: semáforos, monitores, el algoritmo de Dekker y Peterson, los candados.

- **Condición de ocupar y esperar un recurso:** consiste en que un proceso pide un recurso y se le asigna. Antes de soltarlo, pide otro recurso que otro proceso ya tiene asignado.

El problema más serio que se puede presentar en un ambiente de concurrencia es **deadlock**, que consiste en que se presentan al mismo tiempo cuatro condiciones necesarias: la condición de no apropiación, la condición de espera circular, la condición de exclusión mutua y la condición de ocupar y esperar un recurso. Ante esto, si el deadlock involucra a todos los procesos del sistema, el sistema ya no podrá hacer nada productivo. Si el deadlock involucra a algunos procesos, éstos quedarán congelados para siempre.

Las **técnicas para prevenir el deadlock** consisten en proveer mecanismos para evitar que se presente alguna de las cuatro condiciones necesarias del deadlock. Algunas de ellas son:

- **Asignar recursos en orden lineal:** esto significa que todos los recursos están etiquetados con un valor diferente y los procesos solo pueden hacer peticiones de recursos hacia adelante. Esto es, si un proceso tiene el recurso con etiqueta 5 no puede pedir recursos cuya etiqueta sea menor que 5. Con esto se evita la condición de ocupar y esperar un recurso.
- **Asignar todo o nada:** este mecanismo consiste en que el proceso pida todos los recursos que va a necesitar de una vez y el sistema se los dará solamente si puede dárselos todos. De no ser así, no le da nada y lo bloquea.

- **Algoritmo del banquero:** este algoritmo usa una tabla para saber cuántos recursos tiene de todo tipo. También requiere que los procesos informen del máximo de recursos que van a usar de cada tipo. Cuando un proceso pide un recurso, el algoritmo verifica si, asignándole ese recurso, todavía quedan otros del mismo tipo para que alguno de los procesos en el sistema todavía se pueda recibir hasta su máximo. Si la respuesta es afirmativa, el sistema se dice que está en estado seguro y se otorga el recurso. Si la respuesta es negativa, se dice que el sistema está en estado inseguro y se hace esperar a ese proceso.

Para detectar un deadlock, se puede usar el algoritmo del banquero, ya que, aunque no dice que hay un deadlock, sí detecta el estado inseguro que es la antesala del deadlock. Sin embargo, para detectar el deadlock se pueden usar las gráficas de recursos. En ellas se pueden usar cuadrados para indicar procesos y círculos para los recursos, y flechas para indicar si un recurso ya está asignado a un proceso o si un proceso está esperando un recurso.

2.2 Administración de memoria

La memoria es un gran almacén de bytes con una dirección única, de rápido acceso y compartida por la CPU y los dispositivos de E/S, es volátil y su contenido se pierde si el equipo está apagado.

El Sistema Operativo es el responsable de:

- Conocer qué partes de la memoria están siendo utilizadas y por qué procesos.
- Decidir qué procesos se cargarán en memoria cuando se genere el próximo espacio libre.
- Asignar y reclamar espacio de memoria cuando sea necesario.

En esta sección se describirán las técnicas más usuales en el manejo de la memoria, revisando los conceptos relevantes.

Se abarcarán los esquemas de gestión simple de la memoria real, la multiprogramación en memoria real con sus variantes, el concepto de overlay, la multiprogramación con intercambio y los esquemas de gestión de memoria virtual.

Un vistazo al contenido que se va a desarrollar en esta sección se muestra en la figura adjunta. Es una gráfica en donde se especifican, en términos generales, los conceptos más importantes en cuanto a las técnicas empleadas en el manejo de memoria.

Real	Real		Real		Virtual	
Mono Usuario	Multiprogramación		Multiprogramación		Multiprogramación	
	Particionamiento		Paginación Simple	Segmentación Simple	Paginación Virtual	Segmentación Virtual
	Fija	Dinámica	Combinación		Combinación	
	Reubicación, Protección					

Figura 3: Panorama del manejo de memoria

2.2.1 Manejo de memoria en sistemas monousuario sin intercambio

Este esquema se usa principalmente en sistemas monousuario y monotarea, como son los ordenadores personales con DOS. Bajo este esquema, la memoria real se utiliza para almacenar el programa que se esté ejecutando en un momento dado, con la visible desventaja de estar limitado a la cantidad de RAM disponible. La organización física bajo este esquema es muy simple: el sistema operativo se ubica en las localidades superiores o inferiores de la memoria, seguido por algunos manejadores de dispositivos

(drivers). Esto deja un espacio contiguo de memoria disponible que es utilizado por los programas del usuario, dejando generalmente la ubicación de la pila (stack) al último, con el objetivo de que ésta pueda crecer hasta el máximo posible. Estas diferentes opciones se pueden ver en la figura anterior (figura 3). Como es obvio, bajo estos esquemas no se requieren algoritmos sofisticados para asignar la memoria a los diferentes procesos, ya que éstos son ejecutados secuencialmente conforme van terminando los anteriores.

2.2.2 Multiprogramación en memoria real

Con la constatación de que los sistemas de proceso por lotes invertían una gran cantidad de tiempo en operaciones de entrada y salida, donde la intervención de la unidad central de proceso era prácticamente nula, se comenzó a estudiar el modo de hacer que la CPU se mantuviera más tiempo ocupada. Fue así como nació el concepto de multiprogramación, el cual consiste en poner en la memoria física más de un proceso al mismo tiempo, de manera que si el que se está ejecutando en este momento entra en un periodo de entrada/salida, se pueda tomar otro proceso para que haga uso de la unidad central de proceso. De esta forma, la memoria física se dividía en secciones de tamaño suficiente para contener varios programas.

Dentro del esquema de multiprogramación en memoria real surgieron dos problemas interesantes: la protección y la relocalización.

- **El problema de la relocalización**

Este problema no es exclusivo de la multiprogramación en memoria real, se presentó inicialmente aquí, pero también se presenta en los esquemas de memoria virtual. Consiste en que los programas que necesitan cargarse en la memoria real ya están compilados y enlazados, de manera que, internamente, contienen una serie de referencias a direcciones de instrucciones, rutinas y procedimientos que ya no son válidas en el espacio de direcciones de memoria real de la sección en la que se carga el programa. Esto es, cuando se compiló el programa se definieron o resolvieron las direcciones de memoria de acuerdo con la sección de ese momento, pero si el programa se carga en una sección diferente, las direcciones reales ya no coinciden. En este caso, el manejador de memoria puede solucionar el problema de dos maneras: *estática* o *dinámica*. La solución estática consiste en que todas las direcciones del programa se vuelvan a recalcular al momento en que el programa se carga a memoria, esto es, prácticamente se vuelve a recompilar el programa. La solución dinámica consiste en tener un registro que guarde la dirección base de la sección que va a contener al programa. Cada vez que el programa haga una referencia a una dirección de memoria, se le suma el registro base para encontrar la dirección real. Por ejemplo, suponga que el programa se carga en una sección que comienza en la dirección 100. El programa hará referencias a las direcciones 50, 52, 54. Pero el contenido de esas direcciones no es el deseado, sino las direcciones 150, 152 y 154, ya que ahí comienza el programa. La suma de $100 + 50$, etcétera, se hace en tiempo de ejecución. La primera solución resulta más conveniente en el caso de que el programa contenga ciclos y sea largo, ya que consumirá menos tiempo en la resolución inicial que la segunda solución en las resoluciones que tienen lugar durante la ejecución.

- **El problema de la protección.**

Este problema se refiere a que, una vez que un programa ha sido cargado en memoria en algún segmento en particular, el programador puede direccionar (por error o deliberadamente) posiciones de memoria menores que el límite inferior de su programa, o superiores a la dirección mayor; es decir, podría referenciar localidades fuera de su espacio de direcciones. Obviamente, éste es un problema de protección, ya que no es legal leer o escribir en áreas de otros programas.

La solución a este problema también puede ser el uso de un registro base y un registro límite. El registro base contiene la dirección del comienzo de la sección que contiene el programa, mientras que el registro límite contiene la dirección donde termina. Cada vez que el programa hace una referencia a memoria, se comprueba si ésta pertenece al espacio de direcciones situado entre su registro base y su registro límite. Si no es así, se envía un mensaje de error y se aborta el programa.

- Particiones fijas o particiones variables

En el esquema de la multiprogramación en memoria real se manejan dos alternativas para asignar a cada programa su partición correspondiente: particiones de tamaño fijo o particiones de tamaño variable. La alternativa más simple son las particiones fijas. Dichas particiones se crean cuando se enciende el equipo, y permanecen con los tamaños iniciales hasta que el equipo se reinicia. Es una alternativa muy vieja, las particiones se realizaban por el operador analizando los tamaños estimados de los trabajos de todo el día. Por ejemplo, si el sistema contaba con 512 KB de RAM, se podía asignar 64 KB para el sistema operativo, una partición más de 64 KB, otra de 128 KB y una mayor de 256 KB.

Este esquema era muy simple, pero inflexible, ya que, si surgían trabajos urgentes, por ejemplo, de 400 KB, debían esperar a otro día o reparticionar, inicializando el equipo desde cero. La otra alternativa, que surgió después y como necesidad de mejorar la alternativa anterior, fue crear particiones contiguas de tamaño variable. Para esto, el sistema debía mantener ya una estructura de datos suficiente para saber en dónde había huecos disponibles de RAM y de dónde a dónde había particiones ocupadas por programas en ejecución. Así, cuando un programa requería ser cargado a RAM, el sistema analizaba los huecos para saber si había alguno de tamaño suficiente para el programa entrante, si era así, le asignaba el espacio. Si no, intentaba relocalizar los programas existentes con el propósito de hacer contiguo todo el espacio libre y así obtener un hueco de tamaño suficiente. Si aun así el espacio era insuficiente, el programa se bloqueaba y el sistema optaba por otro. Al proceso mediante el que se juntan los huecos se le denomina de compactación.

En relación a los esquemas de particiones fijas y particiones variables surgen varios problemas: ¿en base a qué criterio se elige el mejor tamaño de partición para un programa?; si, por ejemplo, el sistema tiene dos huecos, uno de 18K y otro de 24K para un proceso que desea 15K, ¿cuál se le asigna?

Existen varios **algoritmos** para dar respuesta a la pregunta anterior, los cuales se enumeran y describen a continuación:

- **Primer ajuste:** se asigna el primer hueco que sea mayor que el tamaño deseado.
- **Mejor ajuste:** se asigna el hueco cuyo tamaño exceda en la menor cantidad al tamaño deseado. Requiere una búsqueda exhaustiva.
- **Peor ajuste:** se asigna el hueco cuyo tamaño exceda en la mayor cantidad al tamaño deseado. Requiere también de una búsqueda exhaustiva.
- **El siguiente ajuste:** es igual que el primer ajuste con la diferencia que se deja un apuntador al lugar en donde se asignó el último hueco para realizar la siguiente búsqueda a partir de él.
- **Ajuste rápido:** se mantienen listas enlazadas separadas de acuerdo con los tamaños de los huecos, para así buscarle a los procesos un hueco más rápido en la cola correspondiente.

Otro problema que se vislumbra desde aquí es que, una vez asignado un hueco, por ejemplo con el peor ajuste, puede ser que el proceso requiriera 12 KB y que el hueco asignado fuera de 64 KB, por lo cual el proceso va a desperdiciar una gran cantidad de memoria dentro de su partición. Este problema se conoce como fragmentación interna.

Por otro lado, conforme el sistema va avanzando en el día, finalizando procesos y comenzando otros, la memoria se va configurando como una secuencia contigua de huecos y de lugares asignados, provocando que existan huecos, por ejemplo, de 12 K, 28 K y 30 K, que sumados dan 70 K, pero que no se podría asignar a un proceso que los requiriese en su totalidad, ya que no son localidades contiguas de memoria (a menos que se realice la compactación). Este fenómeno de existencia de huecos no contiguos de memoria se denomina fragmentación externa.

De cualquier manera, la multiprogramación fue un avance significativo para el mejor aprovechamiento de la unidad central de proceso y de la memoria misma, aun dando lugar a la aparición de los problemas de asignación de memoria, protección y relocalización, entre otros.

- Los overlays

Una vez que surgió la multiprogramación, los usuarios comenzaron a explorar la forma de ejecutar grandes cantidades de código en áreas de memoria muy pequeñas, auxiliados por algunas llamadas al sistema operativo. Es así como nacen los overlays.

Esta técnica consiste en que el programador divide lógicamente un programa muy grande en secciones que puedan almacenarse en las particiones de RAM. Al final de cada sección del programa (o en otros lugares necesarios) el programador inserta una o varias llamadas al sistema con el fin de descargar la sección presente en RAM y cargar otra, que en ese momento reside en el disco duro u otro medio de almacenamiento secundario. Aunque esta técnica resultaba eficaz (porque resolvía el problema) no era eficiente (ya que no lo resolvía de la mejor manera). Esta solución requería que el programador tuviera un conocimiento muy profundo del ordenador y de las llamadas al sistema operativo. Otra desventaja era la portabilidad de un sistema a otro: las llamadas cambiaban, los tamaños de particiones también. Resumiendo, con esta técnica se podían ejecutar programas más grandes que las particiones de RAM, donde la división del código corría a cuenta del programador y el control a cuenta del sistema operativo.

- Multiprogramación con memoria virtual

La necesidad cada vez más imperiosa de ejecutar programas grandes, y el crecimiento en potencia de las unidades centrales de proceso, empujaron a los diseñadores de los Sistemas Operativos a implantar un mecanismo para ejecutar automáticamente programas más grandes que la memoria real disponible, esto es, a ofrecer memoria virtual.

La memoria virtual se llama así porque el programador ve una cantidad de memoria mucho mayor que la real. En realidad se trata de la suma de la memoria de almacenamiento primario (memoria real) y una cantidad determinada de almacenamiento secundario (discos). El sistema operativo, en su módulo de manejo de memoria, se encarga de intercambiar programas enteros, segmentos o páginas entre la memoria real y el medio de almacenamiento secundario. Si lo que se intercambia son procesos enteros, se habla entonces de multiprogramación en memoria real, pero si lo que se intercambian son segmentos o páginas, se puede hablar de multiprogramación con memoria virtual.

La memoria virtual se apoya en varias técnicas interesantes para lograr su objetivo. Una de las teorías más fuertes es la del conjunto de trabajo, la cual se refiere a que un programa o proceso no está usando todo su espacio de direcciones en todo momento, sino que existen un conjunto de localidades activas que conforman el conjunto de trabajo. Si se logra que las páginas o segmentos que contienen al conjunto de trabajo estén siempre en RAM, entonces el programa se ejecutará muy bien.

Otro factor importante es el un fenómeno llamado localidad, referido a la tendencia de algunos programas a usar mucho las instrucciones próximas a la instrucción que se está ejecutando actualmente.

- Paginación pura

La paginación pura en el manejo de memoria consiste en que el sistema operativo divide dinámicamente los programas en unidades de tamaño fijo (generalmente múltiplos de 1 kilobyte), los cuales va a intercambiar entre la RAM y el disco. Al proceso de intercambiar páginas, segmentos o programas completos entre RAM y disco se le conoce como intercambio o swapping. En la paginación, se debe cuidar el tamaño de las páginas, ya que si éstas son muy pequeñas, crece el control por parte del sistema operativo y provoca sobrecarga (overhead). Por otro lado, si las páginas son muy grandes, el overhead disminuye, pero entonces puede ocurrir que se desperdicie memoria en procesos pequeños. Debe haber un equilibrio.

Uno de los aspectos más importantes de la paginación, así como de cualquier esquema de memoria virtual, es la forma de traducir una dirección virtual a dirección real. Para explicarlo, véase la figura.

Como se observa, una dirección virtual $v = (b, d)$ está formada por un número de página virtual b y un desplazamiento d . Por ejemplo, si el sistema ofrece un espacio de direcciones virtuales de 64 kilobytes, con páginas de 4 kilobytes y la RAM sólo es de 32 kilobytes, entonces tenemos 16 páginas virtuales y 8 reales. La tabla de direcciones virtuales contiene 16 entradas, una por cada página virtual. En cada entrada o registro de la tabla de direcciones virtuales se almacenan varios datos: si la página está en disco o en memoria, quién es el dueño de la página, si la página ha sido modificada o es de lectura nada más, etc. Pero el dato que nos interesa ahora es el número de página real que le corresponde a la página virtual. Obviamente, de las 16 virtuales, sólo ocho tendrán un valor de control que dice que la página está cargada en RAM, así como la dirección real de la página, denotada en la figura como b' . Por ejemplo, supóngase que para la página virtual número 14 la tabla dice que, efectivamente está cargada y es la página real 2 (dirección de memoria 8192). Una vez encontrada la página real, se le suma el desplazamiento, que es la dirección que deseamos dentro de la página buscada ($b' + d$).

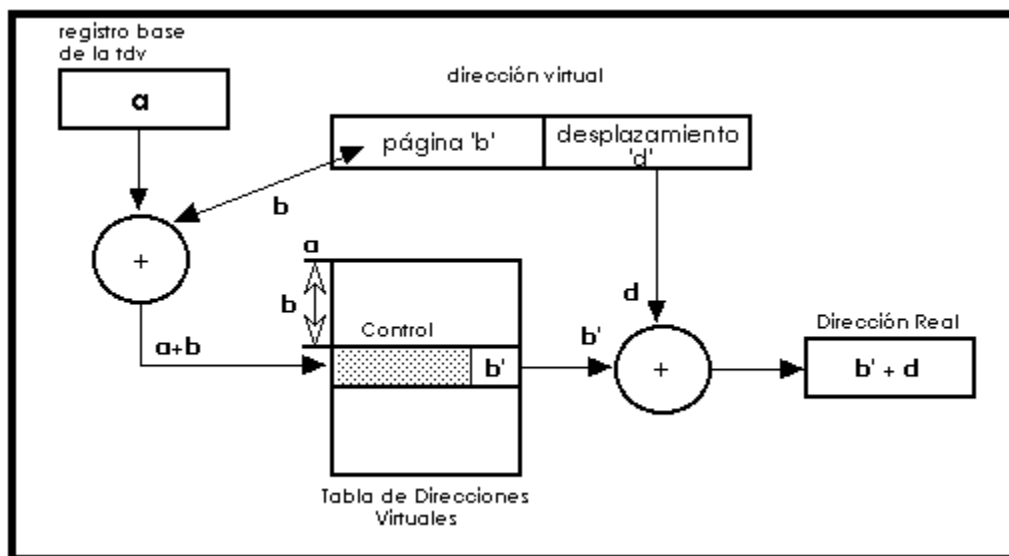


Figura 4: Traducción de direcciones de memoria

La tabla de direcciones virtuales a veces está ubicada en la misma memoria RAM, por lo cual se necesita saber en qué dirección comienza. En este caso, existe un registro con la dirección base, denotada por la letra a en la figura.

Cuando se está buscando una página cualquiera y ésta no está cargada, surge lo que se llama un fallo de página (page fault). Esto resulta costoso para el manejador de memoria, ya que tiene que realizar una serie de pasos extra para poder resolver la dirección deseada y darle su contenido a quien lo pide. Primero, se detecta que la página no está presente y entonces se busca en la tabla la dirección de esta página en el disco. Una vez localizada en el disco se intenta cargar en alguna página libre de RAM. Si no hay páginas libres se tiene que escoger alguna para enviarla hacia el disco. Una vez escogida y enviada al disco, se marca su valor de control en la tabla de direcciones virtuales para indicar que ya no está en RAM, mientras que la página deseada se carga en RAM y se marca su valor para indicar que ahora ya está en RAM.

Todo este procedimiento es caro, ya que se sabe que los accesos a disco duro son del orden de decenas de veces más lentos que a RAM. En el ejemplo anterior se mencionó que cuando se necesita descargar una página de RAM hacia el disco se debe hacer una elección.

Para realizar esta elección existen varios **algoritmos**, los cuales se describen a continuación:

- **Primera en entrar, primera en salir (FIFO):** se escoge la página que haya entrado primero y esté cargada en RAM. Se necesita que en los valores de control se guarde un dato de tiempo. No es eficiente porque no aprovecha ninguna característica de ningún sistema. Es justa e imparcial.
- **La no usada recientemente (NRU):** se escoge la página que no haya sido usada (referenciada) en el ciclo anterior. Pretende aprovechar el hecho de la localidad en el conjunto de trabajo.
- **La usada menos recientemente (LRU):** es parecida a la anterior, pero escoge la página que se usó hace más tiempo, suponiendo que, como lleva mucho sin usarse es muy probable que siga sin usarse en los próximos ciclos. Necesita de una búsqueda exhaustiva.
- **La no usada frecuentemente (NFU):** este algoritmo toma en cuenta no tanto el tiempo, sino el número de referencias. En este caso cualquier página que se use muy poco, menos veces que alguna otra.
- **La menos frecuentemente usada (LFU):** es parecida a la anterior, pero aquí se busca de forma exhaustiva aquella página que se ha usado menos que todas las demás.
- **En forma aleatoria:** elige cualquier página sin aprovechar nada. Es justa e imparcial, pero ineficiente.

Otro dato interesante de la paginación es que ya no se requiere que los programas estén ubicados en zonas de memoria adyacente, ya que las páginas pueden estar ubicadas en cualquier lugar de la memoria RAM.

- Segmentación pura

La segmentación aprovecha el hecho de que los programas se dividen en partes lógicas, como son las partes de datos, de código y de pila (stack). La segmentación asigna particiones de memoria a cada segmento de un programa y busca como objetivos el facilitar la compartición de segmentos (por ejemplo librerías compartidas) y el intercambio entre memoria y los medios de almacenamiento secundario.

Por ejemplo, en la versión de UNIX SunOS 3.5, no existían librerías compartidas para algunas herramientas, por ejemplo, para los editores de texto orientados al ratón y menús. Cada vez que un usuario invocaba a un editor, se tenía que reservar 1 megabyte de memoria. Como los editores son una herramienta muy solicitada y empleada frecuentemente, se dividió en segmentos para la versión 4.x (que a su vez se dividían en páginas), pero lo importante es que la mayor parte del editor es común para todos los usuarios, de manera que la primera vez que cualquier usuario lo invocaba, se reservaba un megabyte de memoria como antes, pero para el segundo, tercero y resto de usuarios, cada editor extra sólo consumía 20 kilobytes de memoria. El ahorro es impresionante. Obsérvese que en la segmentación pura las particiones de memoria son de tamaño variable, en contraste con particiones de tamaño fijo en la paginación pura.

También se puede decir que la segmentación pura tiene una granularidad menor que la paginación, dado el tamaño de segmentos respecto al tamaño de páginas. Nuevamente, para comprender mejor la segmentación, se debe dar un repaso a la forma en que las direcciones virtuales son traducidas a direcciones reales, con ayuda de las figuras.

Prácticamente la traducción es igual que la llevada a cabo en la paginación pura, tomando en consideración que el tamaño de los bloques a controlar por la tabla de traducción es variable, por lo cual cada entrada en dicha tabla debe contener la longitud de cada segmento a controlar. Otra vez se cuenta con un registro base que contiene la dirección del comienzo de la tabla de segmentos. La dirección virtual se compone de un número de segmento (s) y un desplazamiento (d) para ubicar un byte (o palabra) dentro de dicho segmento. Es importante que el desplazamiento no sea mayor que el

tamaño del segmento, lo cual se controla simplemente chequeando que ese valor sea mayor que la dirección del inicio del segmento y menor que el inicio sumado al tamaño.

Una vez dada una dirección virtual $v=(s, d)$, se realiza la operación $b + s$ para hallar el registro (o entrada de la tabla de segmentos) que contiene la dirección de inicio del segmento en la memoria real, denotado por s' . Conociendo la dirección de inicio en memoria real s' sólo resta encontrar el byte o palabra deseada, lo cual se hace sumándole a s' el valor del desplazamiento, de modo que la dirección real $r = s' + d$.

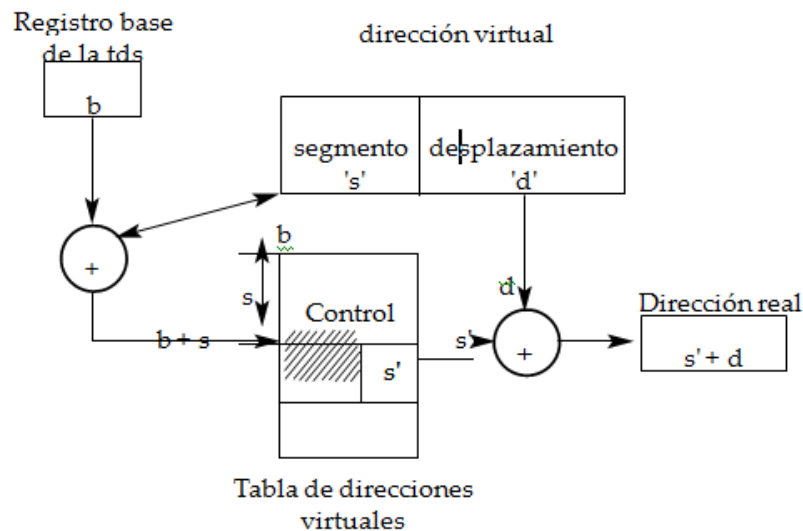


Figura 5: Traducción de direcciones de páginas

Cada entrada en la tabla de segmentos tiene un formato similar al mostrado en la figura de ejemplo. Se tienen campos que indican la longitud, los permisos, la presencia o ausencia y dirección de inicio en memoria real del segmento.

Un hecho notable en los sistemas que manejan paginación es la sucesión de un gran número de fallos de página cuando el proceso comienza a ejecutarse, debido a que se referencian muchas direcciones nuevas por primera vez. Posteriormente el sistema se estabiliza, conforme el número de marcos asignados se acerca al tamaño del conjunto de trabajo.

- Sistemas combinados

La paginación y la segmentación puras son métodos de manejo de memoria bastante efectivos, aunque la mayoría de los Sistemas Operativos modernos implantan esquemas combinados, es decir, combinan la paginación y la segmentación. La idea de combinar estos esquemas se debe a que se aprovechan los conceptos de la división lógica de los programas (segmentos) con la granularidad de las páginas. De esta forma, un proceso estará repartido en la memoria real en pequeñas unidades (páginas), cuya liga son los segmentos. También es factible así el compartir segmentos a medida que las partes necesitadas de los mismos se van referenciando (páginas).

Para comprender este esquema, nuevamente se verá cómo se traduce una dirección virtual en una posición de memoria real. Para la paginación y segmentación puras se puede decir que el direccionamiento es bidimensional porque se necesitan dos valores para hallar la dirección real. Para el caso combinado, se puede decir que se tiene un direccionamiento tridimensional. En la figura de la página siguiente se muestran las partes relevantes para lograr la traducción de direcciones. El sistema debe contar con una tabla de procesos (TP). Por cada renglón de esa tabla se tiene un número de proceso y una dirección a una tabla de segmentos. Es decir, cada proceso tiene una tabla de segmentos. Cuando un proceso hace alguna referencia a memoria, se consulta la TP para encontrar la tabla de segmentos de ese proceso. En cada tabla de segmentos de proceso (TSP) se tienen los números de los segmentos que componen ese proceso. Por cada segmento se tiene una dirección a una tabla de páginas. Cada tabla de páginas tiene las direcciones de las páginas que

componen un solo segmento. Por ejemplo, el segmento 'A' puede estar formado por las páginas reales 'a', 'b', 'c', 'p' y 'x'. El segmento 'B' puede estar compuesto de las páginas 'f', 'g', 'j', 'w' y 'z'.

Para traducir una dirección virtual $v = (s, p, d)$ donde 's' es el segmento, 'p' es la página y 'd' el desplazamiento en la página, se hace lo siguiente: primero se determina de qué proceso es el segmento y se localiza la tabla de segmentos de ese proceso en la TP. Con 's' como índice se encuentra un renglón (registro) en la tabla de segmentos de ese proceso, y en ese renglón está la dirección de la tabla de páginas que componen el segmento. Una vez en la tabla de páginas se usa el valor 'p' como índice para encontrar la dirección de la página en la memoria real. Una vez en esa dirección de memoria real se encuentra el byte (o palabra) requerido por medio del valor de 'd'.

En este esquema pueden generarse dos tipos de fallos: el fallo de página y el fallo de segmento. Cuando se hace referencia a una dirección y el segmento que la contiene no está en RAM (aunque sea parcialmente), se provoca un fallo por falta de segmento y lo que se hace es traerlo del medio de almacenamiento secundario y crearle una tabla de páginas. Una vez cargado el segmento se necesita localizar la página correspondiente, pero ésta no existe en RAM, por lo cual se provoca un fallo de página y se carga de disco, y finalmente se puede ya traer la dirección deseada por medio del desplazamiento de la dirección virtual.

La eficiencia de la traducción de direcciones tanto en paginación pura, como en segmentación pura y en esquemas combinados se mejora usando memorias asociativas para las tablas de páginas y segmentos, así como memorias caché para guardar los mapeos más solicitados.

Otro aspecto importante es la estrategia para cargar páginas (o segmentos) a la memoria RAM. Se usan comúnmente dos estrategias: carga de páginas por demanda y carga de páginas anticipada. La estrategia de carga por demanda consiste en que las páginas solamente son llevadas a RAM si fueron solicitadas, es decir, si se hizo referencia a una dirección situada dentro de ellas. La carga anticipada consiste en tratar de predecir qué páginas serán solicitadas en el futuro inmediato y cargarlas de antemano, para que cuando se pidan ya no ocurran fallos de página. Esa predicción puede ser que aproveche el fenómeno de localidad y que las páginas que se cargan por anticipado sean aquéllas que contienen direcciones contiguas a la dirección que se acaba de referenciar. De hecho, el sistema operativo VMS usa un esquema combinado para cargar páginas: cuando se hace referencia a una dirección cuya página no está en RAM, se provoca un fallo de página y se carga esa página junto con algunas páginas adyacentes. En este caso, la página solicitada se cargó por demanda y las adyacentes se cargaron por anticipación.

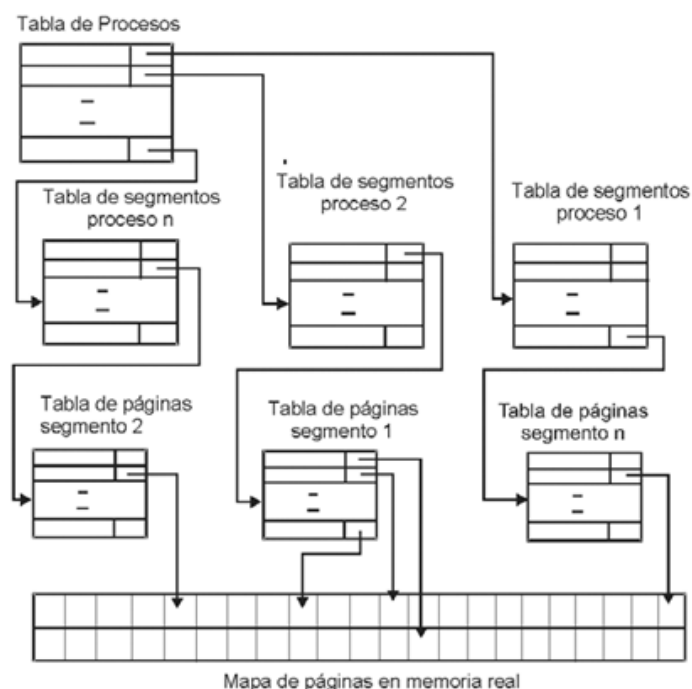


Figura 6: Traducción en la segmentación paginada

2.3 Administración de Entrada / Salida

El código destinado a manejar la entrada y salida de los diferentes periféricos en un sistema operativo es de una extensión considerable, y sumamente complejo. Resuelve las necesidades de sincronizar, atrapar interrupciones y ofrecer llamadas al sistema para los programadores. En esta sección se repasarán los principios más importantes que hay que tomar en cuenta en este módulo del sistema operativo.

2.3.1 Dispositivos de entrada/salida

Los dispositivos de entrada/salida se dividen, en general, en dos tipos: dispositivos orientados a bloque y dispositivos orientados a carácter.

- Los **dispositivos orientados a bloque** tienen la propiedad de que se pueden direccionar, esto es, el programador puede escribir o leer cualquier bloque del dispositivo realizando primero una operación de posicionamiento sobre el mismo. Los dispositivos más comunes orientados a bloque son los discos duros, la memoria, discos compactos y unidades de cinta.
- Los **dispositivos orientados a carácter** son aquéllos que trabajan con secuencias de bytes sin importar su longitud ni ninguna agrupación en especial. No son dispositivos direccionables. Ejemplos de estos dispositivos son el teclado, la pantalla y las impresoras.

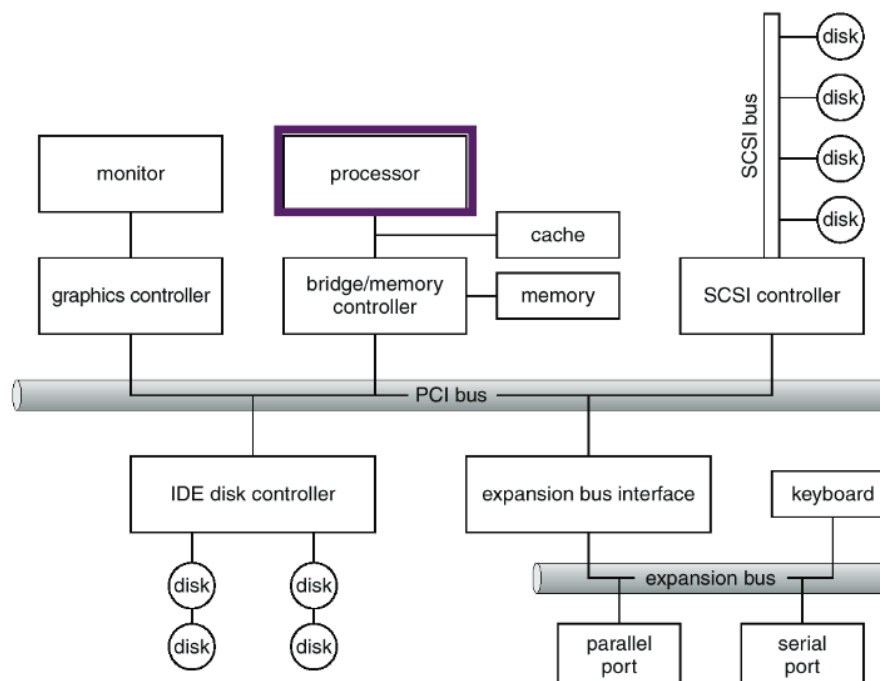


Figura 7: Esquema básico de dispositivos de Entrada/Salida

2.3.2 Controladores de dispositivos (terminales y discos duros)

Los controladores de dispositivos (también llamados adaptadores de dispositivos) son la parte electrónica de los periféricos, los cuales pueden tener la forma de una tarjeta o un circuito impreso integrado en la placa base del ordenador o venderse por separado insertándose en una ranura diseñada al efecto.

Los controladores de dispositivos generalmente trabajan con voltajes de 5 y 12 voltios con el dispositivo propiamente, y con el ordenador a través de interrupciones. Estas interrupciones viajan por el bus del ordenador y son recibidas por la CPU, la cual a su vez pondrá en ejecución algún programa que sabrá qué hacer con esa señal. A ese programa se le llama manejador de dispositivo

(device driver). Algunas veces el mismo controlador contiene un pequeño programa en una memoria de solo lectura o en memoria de acceso aleatorio, no volátil y reescribible, que interactúa con el correspondiente manejador en el ordenador.

Para intercambiar datos o señales entre el ordenador y los controladores, muchas veces se usan registros o secciones predefinidas de la memoria del ordenador. A este esquema se le llama gestión de entrada/salida mapeado por memoria (memory mapped I/O).

2.3.3 Acceso directo a memoria (dma)

El acceso directo a memoria se inventó con el propósito de liberar a la CPU de la carga de atender a algunos controladores de dispositivos. Para comprender su funcionamiento vale la pena revisar cómo trabaja un controlador sin DMA: cuando un proceso requiere algunos bloques de un dispositivo, se envía una señal al controlador con la dirección del bloque deseado. El controlador lo recibe a través del bus, y el proceso puede estar esperando la respuesta (trabajo síncrono) o puede estar haciendo otra cosa (trabajo asíncrono). El controlador recibe la señal y lee la dirección del bus. Envía a su vez una o varias señales al dispositivo mecánico (si es que lo hay) y espera los datos. Cuando los recibe los escribe en un buffer local y envía una señal a la CPU indicándole que los datos están listos. La CPU recibe esta interrupción y comienza a leer byte por byte o palabra por palabra los datos del buffer del controlador (a través del device driver) hasta terminar la operación.

Como se ve, la CPU gasta varios ciclos en leer los datos deseados. El DMA soluciona ese problema de la manera siguiente: cuando un proceso requiere uno o varios bloques de datos, la CPU envía al controlador de DMA la petición junto con el número de bytes deseados y la dirección en dónde quiere que se almacenen de regreso. El DMA actuará como una CPU secundaria dado que tiene el poder de tomar el control del bus e indicarle al verdadero CPU que espere. Cuando el controlador tiene listos los datos, el DMA escucha si el bus está libre aprovechando esos ciclos para ir leyendo los datos del buffer del controlador e ir escribiéndolos en el área de memoria que la CPU le indicó.

Una vez que todos los datos han sido escritos en el destino, se envía una interrupción a la CPU para que los use. El ahorro con el DMA consiste en que la CPU ya no es interrumpida (aunque sí puede ser retardada por el DMA), salvando así el cambio de contexto, y además el DMA aprovechará aquellos ciclos en que el bus no es usado por la CPU.

El hecho de que los controladores necesiten buffers internos se debe a que, conforme ellos reciben datos de los dispositivos que controlan, deben poder almacenarlos temporalmente, pues la CPU no está lista en todo momento para leerlos.

2.3.4 Principios del software de entrada/salida

Los principios de software en la E/S se resumen en cuatro puntos: el software debe ofrecer manejadores de interrupciones, manejadores de dispositivos, software que sea independiente de los dispositivos y software para usuarios.

- **Manejadores de interrupciones.**

El primer objetivo referente a los manejadores de interrupciones consiste en que el programador o el usuario no debe darse cuenta de los manejos de bajo nivel para los casos en que el dispositivo esté ocupado y se deba suspender el proceso o sincronizar algunas tareas. Desde el punto de vista del proceso o usuario, el sistema simplemente tarda más o menos en responder a su petición.

- **Manejadores de dispositivos.**

El sistema debe proporcionar los manejadores de dispositivos necesarios para los periféricos, así como ocultar las peculiaridades del manejo interno de cada uno de ellos, tales como el formato de la información, los medios mecánicos, los niveles de voltaje y otros. Por ejemplo, si el sistema tiene varios tipos diferentes de discos duros las diferencias técnicas existentes entre ellos no deben importar al usuario o programador. Los manejadores le deben ofrecer el mismo conjunto de rutinas para leer y escribir datos.

- **Software independiente del dispositivo.**

Este es un nivel superior de independencia que el ofrecido por los manejadores de dispositivos. Aquí el sistema operativo debe ser capaz, en la medida de lo posible, de ofrecer un conjunto de utilidades para acceder a periféricos o programarlos de una manera consistente. Por ejemplo, que para todos los dispositivos orientados a bloque se disponga de una llamada para decidir si se desea usar buffers o no, o para posicionarse en ellos.

- **Software para usuarios.**

La mayoría de las rutinas de Entrada/Salida trabajan en modo privilegiado o son llamadas al sistema que se ligan a los programas del usuario, formando parte de sus aplicaciones, y que no permiten ninguna flexibilidad en cuanto a la apariencia de los datos. Existen otras librerías en donde el usuario sí tiene poder de decisión (por ejemplo la llamada a printf en el lenguaje C). Otra facilidad ofrecida son las áreas de encolado de trabajos (spooling areas), tales como las de impresión y correo electrónico.

2.3.5 Relojes

Los relojes son esenciales para el buen funcionamiento de cualquier sistema porque juegan un papel decisivo en la sincronización de procesos, en la programación de trabajos por lotes, y para la asignación de turnos de ejecución entre otras tareas relevantes. Generalmente se cuenta con dos relojes en el sistema: uno que lleva la hora y fecha del sistema y que oscila entre 50 y 60 veces por segundo, y otro reloj que oscila entre 5 y 3.000 millones de veces por segundo y que se encarga de enviar interrupciones a la CPU de manera periódica. El reloj de mayor frecuencia sirve para controlar el tiempo de ejecución de los procesos, para despertar los procesos que están durmiendo y para lanzar o iniciar procesos que fueron programados.

Para mantener la hora y fecha del sistema generalmente se usa un registro alimentado por una pila de alta duración que almacena estos datos y que se programan en fábrica por primera vez. Así, aunque se suspenda la energía la fecha permanece. Para lanzar procesos (chequeo de tiempo ocioso de un dispositivo, terminación del time-slice de un proceso, etc.), se almacena un valor en un registro (QUANTUM) el cual se decrementa con cada ciclo del reloj. Cuando llega a cero, se dispara un proceso que ejecutará las operaciones necesarias (escoger un nuevo proceso en ejecución, verificar el funcionamiento del motor del disco flexible, hacer eco de un carácter del teclado, etc.).

2.3.6 Plug and play

Si bien durante años la arquitectura interna de los miniordenadores e incluso la del PC de IBM original, se apoyaban en un solo Bus de comunicaciones, a medida que aumentó la rapidez de los procesadores y las memorias, la capacidad de este bus se sometió a una demanda excesiva. Para corregirlo se añadieron más buses, tanto para los dispositivos de E/S más rápidos, como para el tráfico entre la CPU y la memoria. Como consecuencia de esta evolución, en la actualidad un sistema Pentium grande tiene al menos nueve buses (caché; memoria; PCI; AGP; USB; IDE; SCSI e ISA), todos con diferente tasa de transferencia y función.

Para operar en un entorno tan variado y complejo, el sistema operativo tiene que saber qué dispositivos hay y cómo configurarlos. Este requisito llevó a Intel y Microsoft a diseñar un sistema para PC denominado Plug and Play (Conectar y Usar), basado en un concepto similar al que se implementó por primera vez en la Apple Macintosh.

Antes de Plug and Play, cada tarjeta de E/S tenía un nivel fijo de solicitud de interrupción y direcciones fijas para sus registros de E/S. Los problemas se presentaban cuando un usuario adquiría alguna ampliación (p. ej. una tarjeta de sonido y un modem) y resultaba que ambas coincidían en alguno de sus parámetros de configuración (p. ej. la misma interrupción). La solución fue incluir interruptores o puentes (jumpers) en cada tarjeta de E/S, haciendo que ésta admitiera varias configuraciones diferentes y explicando al usuario cómo ajustarlos para evitar conflictos con otras tarjetas instaladas. Esta situación convertía la instalación y puesta en marcha de nuevas capacidades en un calvario reservado a unos pocos expertos.

Plug and Play (cuyo acrónimo es PnP) vino a poner orden en el caos de la configuración de dispositivos de E/S. Con PnP, el sistema recaba información de manera automática sobre los dispositivos de E/S, asigna de modo centralizado niveles de interrupción y direcciones de E/S y luego comunica a cada tarjeta su asignación, configurándose ésta de acuerdo a lo establecido. En cada arranque del sistema la configuración es susceptible de modificarse si es preciso.

La especificación Plug and Play no es una propiedad o característica exclusiva del sistema operativo. Para que un sistema sea PnP deben estar implicados un conjunto de diferentes colectivos: el suministrador del sistema operativo; el fabricante del hardware del sistema; el desarrollador de la BIOS y los fabricantes de dispositivos. Para conseguir la coordinación necesaria debe existir un conjunto bien definido de interfaces y delegaciones claras de responsabilidad para alcanzar las metas previstas.

Debemos recordar que todo el subsistema PnP concierne principalmente a la gestión de cuatro tipos de recursos diferentes en nombre de los dispositivos:

- o **Memoria:** los requisitos de memoria física del dispositivo (p.ej. cuántas páginas de memoria necesita el dispositivo y cualquier restricción de alineamiento).
- o **E/S:** los puertos de E/S a los que responderá el dispositivo. La información de configuración del dispositivo incluye una especificación de cada uno de los conjuntos alternativos de puertos que el dispositivo puede utilizar (si los hay).
- o **DMA:** cualquier canal DMA que requiera el dispositivo y cualquier canal alternativo que pueda utilizar.
- o **IRQ:** los requisitos de IRQ del dispositivo, IRQ alternativos y si el dispositivo puede o no compartir una IRQ con otros dispositivos.

Merece la pena aclarar que la especificación Plug and Play nació para permitir la configuración de dispositivos conectados en el bus ISA de un ordenador. El bus PCI ya nació comportándose como Plug and Play, aunque la especificación PCI no emplea en ningún momento el término Plug and Play, su hardware implementa lo que, hoy en día, se conoce como Plug and Play.

2.3.7 La interfaz de usuario

Todo ordenador debe disponer de un sistema capaz de interactuar con el usuario del mismo, mediante el cual puedan introducirse órdenes y recibir la información generada de una forma adecuada e inteligible, estableciéndose una comunicación hombre-máquina. Al sistema compuesto por elementos hardware y software adecuados a este fin, se le conoce como la interfaz de usuario.

La interfaz de usuario debe proporcionar la capacidad de comunicación bidireccional. Hoy en día, cualquier ordenador de uso general dispone por lo menos de un teclado y un ratón, como canal de comunicación hombre-máquina y de una pantalla o monitor como canal máquina-hombre.

No sólo eso, sino que actualmente contamos con sistemas que tienen interfaces de usuario mucho más avanzadas, como son pantallas multitáctiles, las tabletas y los Smartphones. Sistemas que permiten la comunicación hombre-máquina con el dedo y una pantalla como canal máquina-hombre.

Toda interfaz hardware se complementa con una interfaz software que aporta en gran medida la funcionalidad y comunicabilidad definitiva, estableciendo una filosofía o modo de interacción con el usuario que puede llegar a ser radicalmente distinta aun empleando los mismos interfaces físicos.

2.4 Sistemas de archivos

Un sistema de archivos (File System) es una estructura de directorios con algún tipo de organización que nos permite almacenar, crear y borrar archivos en diferentes formatos. En esta sección se revisarán conceptos importantes relacionados con los sistemas de archivos.

2.4.1 Almacenamiento físico de datos

En un sistema informático existe la necesidad por parte de los usuarios y aplicaciones de almacenar datos en algún medio, unas veces por periodos largos y otras simplemente por instantes. Cada aplicación y cada usuario deben tener ciertos derechos respecto a sus datos, como son el poder crearlos y borrarlos, o cambiarlos de lugar; así como tener privacidad frente a otros usuarios o aplicaciones. El subsistema de archivos del sistema operativo se debe encargar de estos detalles, además de establecer el formato físico en el cual almacenará los datos en discos duros, cintas o discos flexibles.

Tradicionalmente la información en los sistemas modernos se almacena en **discos duros**, discos flexibles y unidades de disco óptico, y en todos ellos se comparten algunos esquemas básicos para darles formato físico: las superficies de almacenamiento están divididas en círculos concéntricos llamados **pistas**, y cada pista dividida en **sectores**. A la unión lógica de varias pistas a través de varias superficies paralelas de almacenamiento se les llama **cilindros**, los cuales son inspeccionados en el momento de lectura o escritura de datos por las respectivas unidades físicas llamadas cabezas. Las superficies de almacenamiento reciben el nombre de **platos** y generalmente están en movimiento rotativo para que las cabezas accedan a las pistas que los componen. Los datos se escriben a través de los sectores en las pistas y cilindros modificando las superficies por medio de las cabezas.

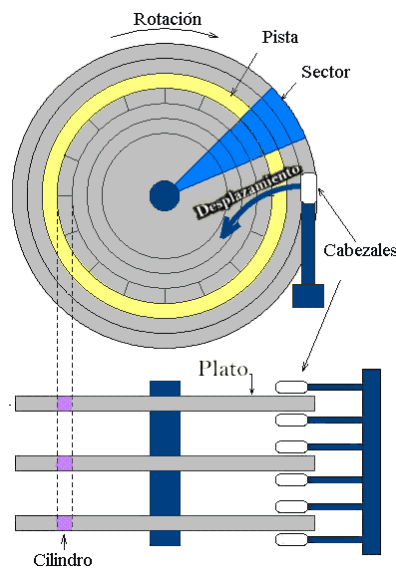


Figura 8: Estructura del disco duro que muestra: pista, sector, cabezales y plato

Al tiempo que una cabeza necesita para ir de una pista a otra se le llama **tiempo de búsqueda** y dependerá de la distancia entre la posición actual y la distancia a la pista buscada. El tiempo que tarda una cabeza en posicionarse dentro de una pista, desde el sector actual al sector deseado se le denomina **tiempo de latencia** y depende de la distancia entre sectores y la velocidad de rotación del disco. El impacto que tienen las lecturas y escrituras sobre el sistema está determinado por la tecnología usada en los platos y cabezas y por la forma de resolver las peticiones de lectura y escritura, es decir, los algoritmos de planificación.

Los **algoritmos de planificación de peticiones de lectura y escritura** a discos se encargan de registrar dichas peticiones y de responderlas en un tiempo razonable. Los algoritmos más comunes para esta tarea son:

- **Primero en llegar, primero en ser servido (FIFO):** las peticiones son encoladas de acuerdo con el orden en que llegaron y de esa misma forma se van leyendo o escribiendo las mismas. La ventaja de este algoritmo es su simplicidad que no causa sobrecarga, y su desventaja principal es que no aprovecha ninguna característica de las peticiones, de manera que es muy factible que el brazo del disco se mueva muy ineficientemente, ya que las peticiones pueden tener direcciones en el disco unas muy alejadas de otras. Por ejemplo, si se están haciendo peticiones a los sectores 6, 10, 8, 21 y 4, las mismas serán resueltas en el mismo orden.
- **Primero el más cercano a la posición actual:** en este algoritmo las peticiones se ordenan de acuerdo con la posición actual de la cabeza lectora, sirviendo primero a aquellas peticiones más cercanas y reduciendo, así, el movimiento del brazo, lo cual constituye la ventaja principal de este algoritmo. Su desventaja consiste en que puede haber solicitudes que se queden esperando para siempre, en el infortunado caso de que existan peticiones muy alejadas y en todo momento estén entrando peticiones que estén más cercanas. Para las peticiones 6, 10, 8, 21 y 4, las mismas serán resueltas en el orden 4, 6, 8, 10 y 21.
- **Por exploración (algoritmo del elevador):** en este algoritmo el brazo se está moviendo en todo momento desde el perímetro del disco hacia su centro y viceversa, resolviendo las peticiones que existan en la dirección que tenga en turno. En este caso las peticiones 6, 10, 8, 21 y 4 serán resueltas en el orden 6, 10, 21, 8 y 4; es decir, la posición actual es 6 y como va hacia los sectores de mayor numeración (hacia el centro, por ejemplo), en el camino sigue el sector 10, luego el 21, siendo el más céntrico. Ahora el brazo resolverá las peticiones en su camino hacia el borde del plato y la primera petición que se encuentra es la del sector 8 y luego la 4. La ventaja de este algoritmo es que el brazo se mueve mucho menos que en FIFO y evita la espera indefinida; su desventaja es que no es justo, ya que no sirve las peticiones en el orden en que llegan, además las peticiones en los extremos interior y exterior gozan de un tiempo de respuesta un poco mayor.
- **Por exploración circular:** es una variación del algoritmo anterior, con la única diferencia que, al llegar a la parte central, el brazo regresa al exterior sin resolver ninguna petición, lo cual proporciona un tiempo de respuesta más cercano al promedio para todas las peticiones, sin importar si están cercas del centro o del exterior.

2.4.2 Asignación del espacio de almacenamiento

El subsistema de archivos se debe encargar de localizar espacio libre en los medios de almacenamiento para guardar archivos y para después borrarlos, renombrarlos o agrandarlos. Para ello se vale de ubicaciones especiales que contienen la lista de archivos creados y por cada archivo una serie de direcciones que apuntan a su contenido. Esas localidades especiales se llaman directorios. Para asignarle espacio a los archivos existen tres criterios generales que se describen a continuación.

- **Asignación contigua:** cada directorio contiene los nombres de archivos y la dirección del bloque inicial de cada archivo, así como el tamaño total de cada uno. Por ejemplo, si un archivo comienza en el sector 17 y mide 10 bloques, cuando el archivo sea accedido, el brazo se moverá inicialmente al bloque 17 y de ahí hasta el 27. Si el archivo es borrado y luego es creado otro más pequeño, quedarán huecos inútiles entre archivos útiles, lo cual se denomina fragmentación externa.
- **Asignación encadenada:** con este criterio los directorios contienen los nombres de archivos y por cada uno de ellos la dirección del bloque inicial que compone al archivo. Cuando un archivo es leído, el brazo va a esa dirección inicial y encuentra los datos iniciales junto con la dirección del siguiente bloque y así sucesivamente.

Con este criterio no es necesario que los bloques estén contiguos y no existe la fragmentación externa, pero en cada eslabón de la cadena se desperdicia espacio con las direcciones mismas. En otras palabras, lo que se crea en el disco es una lista ligada.

- **Asignación con índices (indexada):** en este esquema se guarda en el directorio un bloque de índices para cada archivo, con apuntadores hacia todos sus bloques constituyentes, de manera que el acceso directo se agiliza notablemente, a cambio de sacrificar varios bloques para almacenar dichos apuntadores. Cuando se quiere leer un archivo o cualquiera de sus partes, se hacen dos accesos: uno al bloque de índices y otro a la dirección deseada. Éste es un esquema excelente para archivos grandes pero no para los pequeños, para los que la relación entre bloques destinados a índices respecto de los destinados para datos supone un coste inasumible.

2.4.3 Métodos de acceso en los sistemas de archivos

Los métodos de acceso se refieren a las capacidades que el subsistema de archivos proporciona para acceder a datos dentro de los directorios y medios de almacenamiento en general. Existen tres formas generales: acceso secuencial, acceso directo y acceso directo indexado.

- **Acceso secuencial:** es el método más lento y consiste en recorrer los componentes de un archivo uno a uno hasta llegar al registro deseado. Se necesita que el orden lógico de los registros sea igual al orden físico en el medio de almacenamiento. Este tipo de acceso se usa comúnmente en cintas y cartuchos.
- **Acceso directo:** permite acceder a cualquier sector o registro inmediatamente, por medio de llamadas al sistema como la de seek. Este tipo de acceso es rápido y se usa comúnmente en discos duros y discos o archivos manejados en memoria de acceso aleatorio.
- **Acceso directo indexado:** este tipo de acceso es útil para grandes volúmenes de información o datos. Consiste en que cada archivo tiene una tabla de apuntadores, cada de los cuales va a la dirección de un bloque de índices, lo cual permite que el archivo se expanda a través de un espacio enorme. Consume una cantidad importante de recursos en las tablas de índices, pero es muy rápido.

2.4.4 Operaciones soportadas por el subsistema de archivos

Independientemente de los algoritmos de asignación de espacio, de los métodos de acceso y de la forma de resolver las peticiones de lectura y escritura, el subsistema de archivos debe proveer un conjunto de llamadas al sistema para operar con los datos proporcionando mecanismos de protección y seguridad.

Las operaciones básicas que la mayoría de los sistemas de archivos soportan son:

- **Crear (create):** permite crear un archivo sin datos, con el propósito de indicar que ese nombre ya está usado. Se deben crear las estructuras básicas para soportarlo.
- **Borrar (delete):** eliminar el archivo y liberar los bloques para su uso posterior.
- **Abrir (open):** antes de usar un archivo se debe abrir para que el sistema conozca sus atributos, tales como el propietario, la fecha de modificación, etc.
- **Cerrar (close):** después de realizar todas las operaciones deseadas, el archivo debe cerrarse para asegurar su integridad y para liberar recursos de su control en la memoria.
- **Leer o Escribir (read, write):** añadir información al archivo o leer el carácter o una cadena de caracteres a partir de la posición actual.
- **Concatenar (append):** es una forma restringida de la llamada write, en la cual sólo se permite añadir información al final del archivo.
- **Localizar (seek):** para los archivos de acceso directo se permite posicionar el apuntador de lectura o escritura en un registro aleatorio, a veces a partir del inicio o final del archivo.

- **Leer atributos:** permite obtener una estructura con todos los atributos del archivo especificado, tales como permisos de escritura, de borrado, ejecución, etc.
- **Poner atributos:** permite cambiar los atributos de un archivo. Por ejemplo en UNIX, donde todos los dispositivos se manejan como si fueran archivos, es posible cambiar el comportamiento de un terminal con una de estas llamadas.
- **Renombrar (rename):** permite cambiar el nombre e incluso a veces la posición en la organización de directorios del archivo especificado.

Los subsistemas de archivos también proporcionan un conjunto de llamadas para operar sobre directorios, las más comunes son crear, borrar, abrir, cerrar, renombrar y leer. Sus funcionalidades son obvias, pero existen también otras dos operaciones no tan comunes que son la de crear un enlace y la de destruir un enlace. La operación de crear un enlace sirve para que, desde diferentes puntos de la organización de directorios, se pueda acceder a un mismo archivo sin necesidad de copiarlo o duplicarlo. La llamada a destruir un enlace lo que hace es eliminar esas referencias, sin afectar al archivo original.

2.5 Conclusión

En este segundo apartado se han tratado los componentes de los Sistemas Operativos en profundidad, que son:

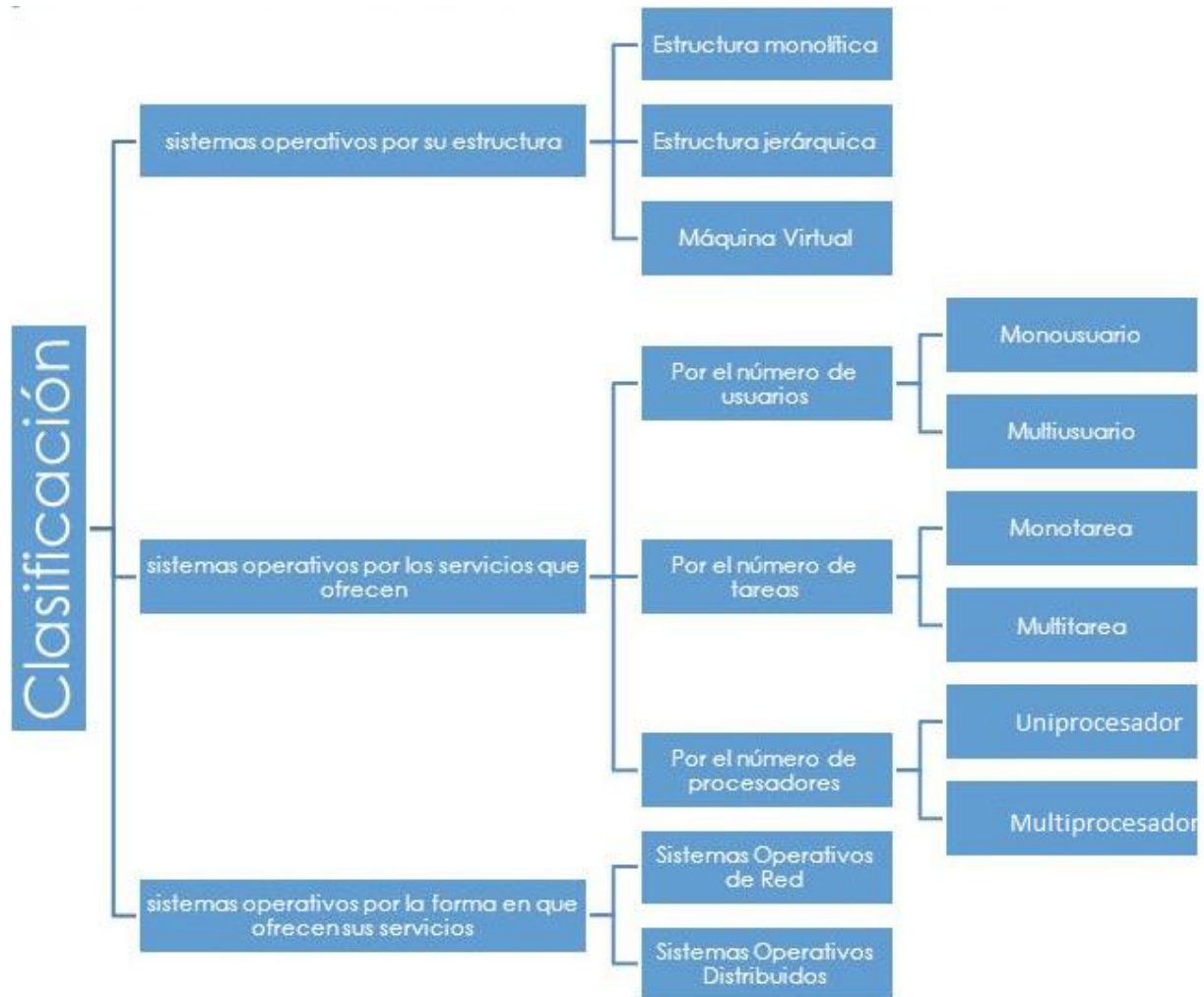
- Administrador de procesos
- Administrador de memoria principal
- Administrador del sistema de E/S
- Administrador de archivos

Y como hemos visto, estos componentes, llevan a cabo los cuatro cometidos que tiene encomendado un Sistema Operativo:

- La gestión de procesos.
- La gestión de memoria.
- El sistema de archivos.
- La gestión de la Entrada/Salida.

3 Clasificación de los Sistemas Operativos

En esta sección se describirán las características que clasifican a los Sistemas Operativos, mostrando varias clasificaciones básicas



3.1 Sistemas Operativos por su capacidad

Esta clasificación es la más común y la más conocida desde el punto de vista del usuario final, y se comprende fácilmente con el cuadro sinóptico que a continuación se muestra.

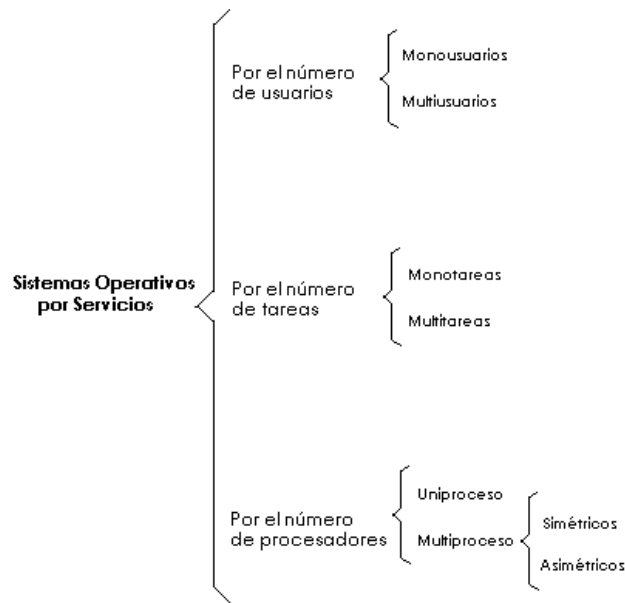


Figura 9: Clasificación de Sistemas Operativos por su capacidad

- **Monousuario:** los Sistemas Operativos monousuarios son aquéllos que soportan un único usuario a la vez, sin importar el número de procesadores que tenga el ordenador o el número de procesos o tareas que el usuario pueda ejecutar en un mismo instante de tiempo. Los ordenadores personales típicamente se han clasificado en esta categoría.
- **Multiusuario:** los Sistemas Operativos multiusuarios son capaces de dar servicio a más de un usuario a la vez, ya sea por medio de varias terminales conectadas al ordenador, o por medio de sesiones remotas en una red de comunicaciones. No importa el número de procesadores en la máquina ni el número de procesos que cada usuario pueda ejecutar simultáneamente.
- **Monotarea:** los sistemas monotarea son aquéllos que sólo permiten una tarea a la vez por usuario. Puede darse el caso de un sistema multiusuario y monotarea, en el cual se admiten varios usuarios al mismo tiempo, pero cada uno de ellos puede estar haciendo sólo una tarea a la vez.
- **Multitarea:** un sistema operativo multitarea es aquél que permite al usuario estar realizando varias labores al mismo tiempo. Por ejemplo, puede estar editando el código fuente de un programa durante su depuración mientras compila otro programa, a la vez que está recibiendo correo electrónico en un proceso desatendido (en segundo plano o background). Es común encontrar en ellos interfaces gráficas orientadas al uso de menús y el ratón, lo cual permite un rápido intercambio entre las tareas para el usuario, mejorando su productividad.
- **Uniprocetador:** un sistema operativo uniprocetador es aquél que es capaz de manejar solamente un procesador del ordenador, de manera que si el ordenador tuviese más de uno le resultaría inútil. Los ejemplos más típicos de este tipo de sistemas son el DOS y el MacOS.

- **Multiprocesador:** un sistema operativo multiprocesador se aplica en ordenadores con dos o más procesadores, siendo capaz de usarlos todos para distribuir su carga de trabajo. Generalmente estos sistemas trabajan de dos formas: simétrica o asimétrica. Cuando se trabaja de manera asimétrica, el sistema operativo selecciona a uno de los procesadores como procesador maestro, el cual servirá como pivote para distribuir la carga a los demás procesadores, que reciben el nombre de esclavos. Cuando se trabaja de manera simétrica, los procesos o partes de ellos (hebras o threads) son enviados indistintamente a cualquiera de los procesadores disponibles, dando lugar bajo este esquema, y teóricamente, a una mejor distribución y un mayor equilibrio en la carga de trabajo.

3.2 Sistemas Operativos por su estructura

A continuación se describen las distintas estructuras que presentan los actuales Sistemas Operativos.

3.2.1 Estructura monolítica

Es la estructura de los primeros Sistemas Operativos, constituidos fundamentalmente por un único programa, compuesto de un conjunto de rutinas entrelazadas de forma que cada una puede llamar a cualquier otra. Las características fundamentales de este tipo de estructura son la construcción del programa final a base de módulos compilados separadamente que se unen a través del enlazador (linkado); y una buena definición de parámetros de enlace entre las distintas rutinas existentes, que puede provocar un gran acoplamiento.

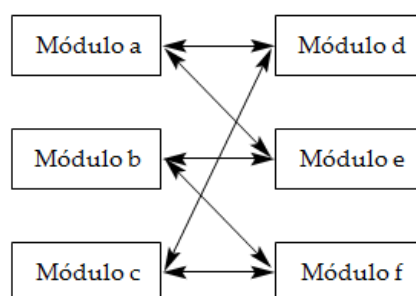


Figura 10: Estructura monolítica

Carecen de protecciones y privilegios al entrar a rutinas que manejan diferentes aspectos de los recursos del ordenador como memoria, disco, etc.

Generalmente están hechos a medida, circunstancia que les confiere eficiencia y rapidez en su ejecución y gestión, pero que también les resta flexibilidad para soportar diferentes ambientes de trabajo o distintos tipos de aplicaciones.

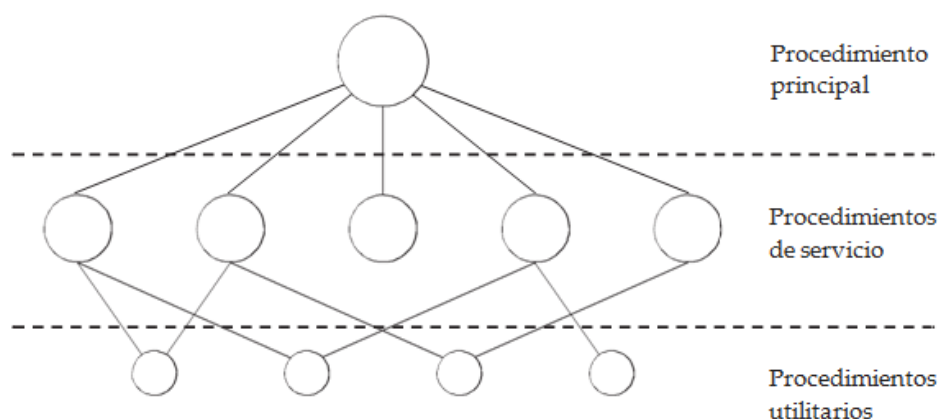


Figura 11: Modelo de Estructuración simple para un Sistema Operativo monolítico

En este tipo de sistemas es posible tener al menos un poco de estructura, derivada del modo en que se llevan a cabo las llamadas al Sistema: los servicios que presta el sistema operativo se solicitan colocando los parámetros en un lugar definido (la pila), y ejecutando después una instrucción de llamada (TRAP). La instrucción cambia de modo usuario a modo kernel y transfiere el control al sistema operativo. Entonces, el sistema operativo obtiene los parámetros y determina qué llamada al sistema debe ejecutarse. Esta organización sugiere una estructura básica en donde un procedimiento principal invoca el procedimiento de servicio solicitado, el cual puede hacer uso de uno o varios procedimientos utilitarios que realizan tareas comunes a más de un procedimiento de servicio. Esta división de los procedimientos en tres capas se muestra en la figura anterior.

3.2.2 Estructura jerárquica o en capas

A medida que fueron creciendo las necesidades de los usuarios y se perfeccionaron los sistemas, se hizo necesaria una mayor organización del software del sistema operativo. Se dividió el sistema operativo en pequeñas partes, de tal forma que cada una de ellas estuviera perfectamente definida y con un claro interfaz con el resto de elementos.

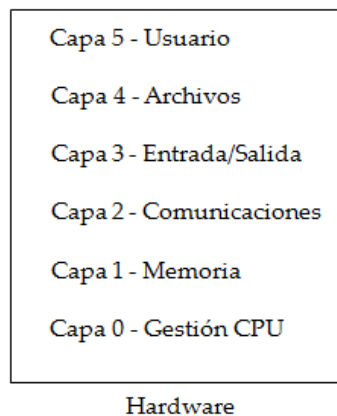


Figura 11: Sistema jerárquico

Se constituyó una estructura jerárquica o de niveles en los Sistemas Operativos, el primero de los cuales fue denominado THE (Technische Hogeschool, Eindhoven), de Dijkstra, que se utilizó con fines didácticos. Se puede pensar también en estos sistemas como si fueran multicapa. Multics y Unix se encuadran en esa categoría.

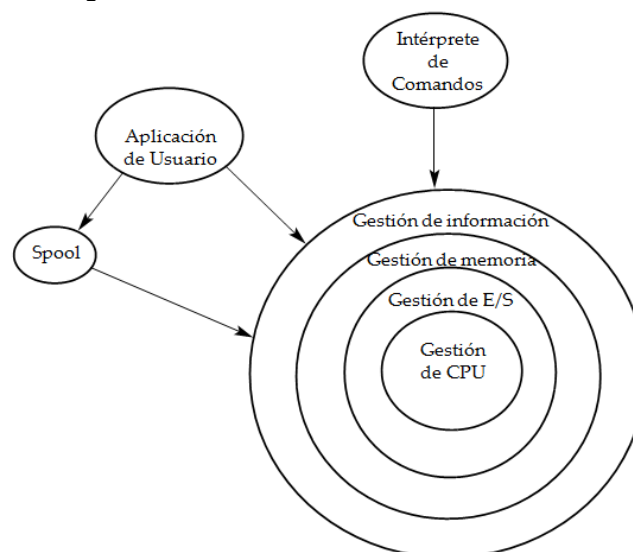


Figura 12: Organización jerárquica (anillos)

En la estructura anterior se basan prácticamente la mayoría de los Sistemas Operativos actuales.

Otra forma de plantear este tipo de sistema es la denominada de anillos concéntricos o rings. En el sistema de anillos cada uno tiene una apertura, conocida como puerta o trampa (trap), por donde pueden entrar las llamadas de las capas inferiores. De esta forma, las zonas más internas del sistema operativo o núcleo del sistema estarán más protegidas de accesos indeseados desde las capas más externas. Las capas más internas serán, por tanto, más privilegiadas que las externas.

3.2.3 Máquina Virtual

Se trata de un tipo de Sistemas Operativos que presentan una interfaz a cada proceso, mostrando una máquina que parece idéntica a la máquina real subyacente. Estos Sistemas Operativos separan dos conceptos que suelen estar unidos en el resto de sistemas: la multiprogramación y la máquina extendida. El objetivo de los Sistemas Operativos de máquina virtual es el de integrar distintos Sistemas Operativos dando la sensación de ser varias máquinas diferentes. El núcleo de estos Sistemas Operativos se denomina monitor virtual y tiene como misión llevar a cabo la multiprogramación, presentando a los niveles superiores tantas máquinas virtuales como se soliciten. Estas máquinas virtuales no son máquinas extendidas, sino una réplica de la máquina real, de manera que en cada una de ellas se pueda ejecutar un sistema operativo diferente, que será el que ofrezca la máquina extendida al usuario.

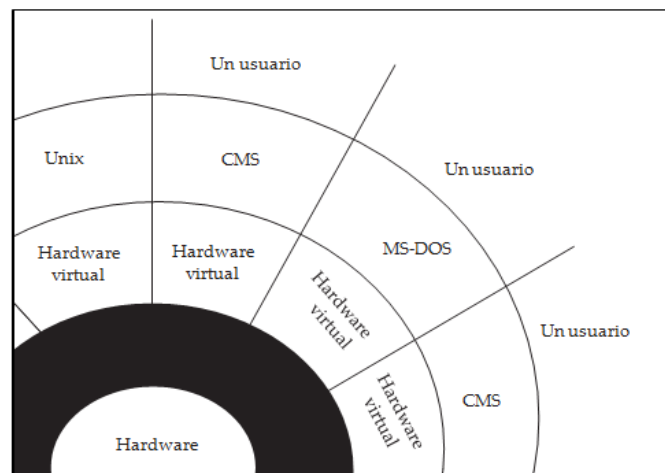


Figura 13: Máquina Virtual

3.2.4 Cliente-servidor (Microkernel)

El tipo más reciente de Sistemas Operativos es el denominado Cliente-servidor, que puede ser ejecutado en la mayoría de los ordenadores, ya sean grandes o pequeños. Este sistema sirve para toda clase de aplicaciones, por tanto, es de propósito general y cumple con las mismas actividades que los Sistemas Operativos convencionales.

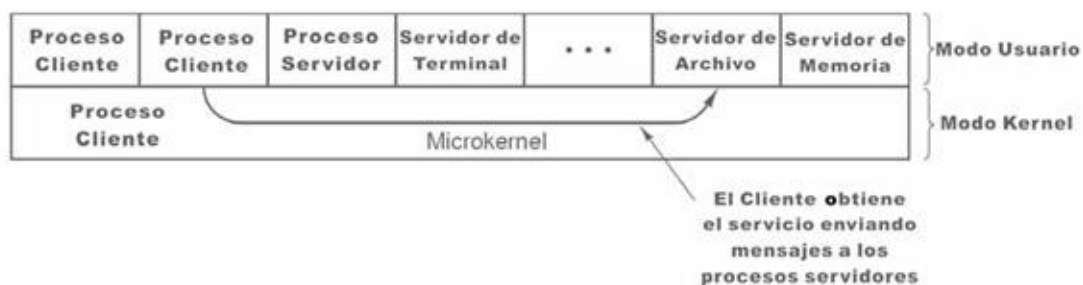


Figura 14: Microkernel

El núcleo tiene como misión establecer la comunicación entre los clientes y los servidores. Los procesos pueden ser tanto servidores como clientes. Por ejemplo, un programa de aplicación

normal es un cliente que llama al servidor correspondiente para acceder a un archivo o realizar una operación de entrada/salida sobre un dispositivo concreto. A su vez, un proceso cliente puede actuar como servidor para otro. Este paradigma ofrece gran flexibilidad en cuanto a los servicios posibles en el sistema final, ya que el núcleo provee solamente funciones muy básicas de memoria, entrada/salida, archivos y procesos, dejando a los servidores proveer la mayor parte de los servicios que el usuario final o programador puede usar. Estos servidores deben tener mecanismos de seguridad y protección que, a su vez, serán filtrados por el núcleo que controla el hardware. Actualmente, se está trabajando en una versión de UNIX que contempla en su diseño este paradigma.

3.2.5 Híbrido

Básicamente es un Sistema Operativo con un núcleo que tiene una arquitectura que trata de combinar aspectos y beneficios de las arquitecturas de los Sistemas Operativos microkernel y kernel monolítico.

La idea detrás de un núcleo híbrido es tener una estructura similar a la que tiene un microkernel, pero implementar dicha estructura según un kernel monolítico. A diferencia de un microkernel, en un núcleo híbrido todos los servicios de Sistema Operativo se encuentran en el espacio del núcleo. Así que no cuenta con ningún beneficio del microkernel, que tiene los servicios en espacio de usuario. Sin embargo, cuenta con los beneficios de los núcleos monolíticos, que no hay sobrecarga de rendimiento en el intercambio de mensajes y el cambio de contexto entre el núcleo y el modo de usuario, cosa que ocurre siempre en el caso de los microkernel.

3.3 Sistemas Operativos por la forma de ofrecer sus servicios.

Esta clasificación también se refiere a una visión externa, la del usuario y su forma de acceder a los servicios. Bajo esta clasificación se pueden detectar dos tipos principales: Sistemas Operativos de red y Sistemas Operativos distribuidos.

3.3.1 Sistemas Operativos de Red.

Los Sistemas Operativos de red se definen como aquéllos que tienen la capacidad de interactuar con Sistemas Operativos en otras computadoras a través de un medio de transmisión con el objeto de intercambiar información, transferir archivos, ejecutar comandos remotos y un sinnúmero de actividades añadidas. El punto crucial de estos sistemas radica en que el usuario debe conocer la sintaxis de un conjunto de comandos o llamadas al sistema para ejecutar estas operaciones, además de tener constancia de la ubicación de los recursos a los que desee acceder.

3.3.2 Sistemas Operativos Distribuidos.

Los Sistemas Operativos distribuidos abarcan los mismos servicios que los Sistemas Operativos de red, logrando integrar recursos (impresoras, unidades de respaldo, memoria, procesos, unidades centrales de proceso) en una sola máquina virtual a la que el usuario accede de forma transparente.

En la realidad no se ha conseguido crear un Sistema Operativo distribuido completamente funcional, por la complejidad que supone afrontar una serie de problemas:

- Distribuir los procesos en las varias unidades de procesamiento.
- Reintegrar sub-resultados.
- Resolver problemas de concurrencia y paralelismo.
- Recuperarse de fallas de algunos recursos distribuidos.
- Consolidar la protección y la seguridad entre los diferentes componentes del sistema y los usuarios.

Los avances tecnológicos en las redes de área local y la aparición de microprocesadores de 32 y 64 bits han conseguido que ordenadores más o menos baratos tengan la suficiente potencia, de forma autónoma, para desafiar en cierto grado a los mainframes. La posibilidad de intercomunicarlos ha permitido partir procesos de cálculo intensivo en unidades más pequeñas, distribuyéndolas entre varios microprocesadores, para luego reunir los sub-resultados creando así una máquina virtual en la red que excede en poder a un mainframe

Al sistema integrador de los microprocesadores que presentan las distintas memorias, procesadores, y todos los demás recursos como una sola entidad en forma transparente se le denomina sistema operativo distribuido. Las razones para crear o adoptar sistemas distribuidos se justifican principalmente bien por la necesidad (debido a que los problemas para resolver son inherentemente distribuidos), bien porque se desea tener más confiabilidad y disponibilidad de recursos.

- **Ventajas de los sistemas distribuidos.**

En general, los sistemas distribuidos (no solamente los Sistemas Operativos) exhiben algunas ventajas sobre los sistemas centralizados, que se describen a continuación.

- *Economía*: el cociente precio/desempeño de la suma del poder de los procesadores separados comparado con el poder de un sistema centralizado es superior cuando están distribuidos.
- *Velocidad*: relacionado con el punto anterior, la velocidad sumada es muy superior.
- *Confiabilidad*: si una sola máquina falla, el sistema total sigue funcionando.
- *Crecimiento*: el poder total del sistema puede irse incrementando al añadir pequeños sistemas, lo cual es mucho más difícil en un sistema centralizado y caro.
- *Distribución*: algunas aplicaciones requieren de por sí una distribución física.

Por otro lado, los sistemas distribuidos también exhiben algunas ventajas sobre sistemas aislados. Estas ventajas son:

- *Compartir datos*: un sistema distribuido permite compartir datos más fácilmente que los sistemas aislados, que tendrían que duplicarlos en cada nodo para lograrlo.
- *Compartir dispositivos*: un sistema distribuido permite acceder a dispositivos desde cualquier nodo en forma transparente, lo cual es imposible con los sistemas aislados. El sistema distribuido logra un efecto sinérgico.
- *Comunicaciones*: la comunicación persona a persona es factible en los sistemas distribuidos, en los sistemas aislados no.
- *Flexibilidad*: la distribución de las cargas de trabajo es factible en los sistemas distribuidos, se puede incrementar el poder de cómputo.

- **Desventajas de los sistemas distribuidos.**

Así como los sistemas distribuidos exhiben grandes ventajas, también se pueden identificar ciertas desventajas, algunas de ellas tan serias que han frenado la producción comercial de estos Sistemas Operativos en la actualidad. El problema más importante surgido en la creación de sistemas distribuidos tiene origen en el software: la compartición de datos y recursos es tan compleja que los mecanismos que la solucionan generan una gran sobrecarga al sistema, haciéndolo ineficiente. Comprobar, por ejemplo, quiénes tienen acceso a algunos recursos y quiénes no, o aplicar los mecanismos de protección y registro de permisos, son tareas que consumen demasiados recursos. En general, las soluciones presentes para estos problemas están aún en una fase incipiente de desarrollo.

Otros problemas de los Sistemas Operativos distribuidos surgen debido a la **conurrencia** y al **paralelismo**.

Tradicionalmente, las aplicaciones se crean para ordenadores de ejecución secuencial, de manera que identificar secciones de código paralelizables es un trabajo arduo, pero necesario para dividir un proceso grande en subprocesos y enviarlos a diferentes unidades de procesamiento para su distribución. Con la concurrencia se deben implantar mecanismos para evitar las condiciones de

competencia, las postergaciones indefinidas, el ocupar un recurso y estar esperando otro, las condiciones de espera circulares y, finalmente, los abrazos mortales (deadlocks).

Estos problemas de por sí se presentan en los Sistemas Operativos multiusuario o multitarea, y su tratamiento en los sistemas distribuidos es aún más complicado y, por lo tanto, necesitará de algoritmos más complejos con la inherente sobrecarga.

3.4 Sistemas Operativos por el entorno hardware en el que se ejecutan

Toda esta historia y desarrollo han dejado una amplia variedad de Sistemas Operativos, muchos de los cuales no se conocen de forma extensa. Veamos una breve referencia a algunos de ellos.

3.4.1 Sistemas Operativos de mainframe

En el extremo superior de los ordenadores se hallan los mainframes, ordenadores gigantes que todavía se encuentran en importantes centros de datos corporativos. Se distinguen de los ordenadores personales fundamentalmente por su capacidad de E/S.

Los Sistemas Operativos de mainframe están orientados al procesamiento de varios trabajos a la vez, con unos elevados requerimientos de E/S. Suelen ofrecer servicios de tres tipos: por lotes, procesamiento de transacciones y tiempo compartido. Un sistema por lotes procesa trabajos rutinarios sin que haya un usuario interactivo presente. El proceso de transacciones maneja solicitudes pequeñas, pero en un número muy elevado (miles o decenas de miles por segundo). El tiempo compartido permite a múltiples usuarios ejecutar trabajos en el ordenador, de forma simultánea, dando la impresión de que se dispone del mismo en exclusividad.

Generalmente estos sistemas han sido propietarios y ligados muy íntimamente con la arquitectura hardware que los sustenta.

3.4.2 Sistemas Operativos de servidor

Después, se identifican los Sistemas Operativos que se ejecutan en servidores, que son ordenadores personales muy grandes o estaciones de trabajo. Dan servicio a múltiples usuarios a través de una red, permitiéndoles compartir recursos hardware y software. Los servidores presentan a los sistemas clientes servicios disponibles como impresión, archivo o Web. Entre los Sistemas Operativos típicos están UNIX, Windows Server (2003, 2008 R2, 2012) y Linux que poco a poco va ganando terreno en el mercado de Sistemas Operativos para uso profesional.

3.4.3 Sistemas Operativos multiprocesador

Es cada vez más común conseguir aumentar la potencia de proceso, mediante la interconexión de varias UCP en un solo sistema. Dependiendo del modo de hacerlo, hablamos de ordenadores paralelos, multiprocesadores, sistemas en grid, etc. Estas disposiciones necesitan de Sistemas Operativos especiales, pero con frecuencia éstos son variaciones de los Sistemas Operativos de servidor, con funciones especiales para comunicación y conectividad.

3.4.4 Sistemas Operativos de PC

Estos Sistemas Operativos, los más comúnmente conocidos, se especializan en presentar una buena interfaz a un solo usuario. Se emplean ampliamente para las labores personales y profesionales de los usuarios como puede ser procesamiento de texto, hojas de cálculo, pequeñas bases de datos,

manejo de gráficos sencillos. La familia de sistemas Windows de Microsoft, el sistema operativo Mac OS y Linux son los ejemplos más representativos de Sistemas Operativos en este segmento.

3.4.5 Sistemas Operativos de tiempo real

Este tipo de Sistemas Operativos se caracteriza porque su parámetro clave es el tiempo. Empleado principalmente en sistemas de control de procesos industriales, los ordenadores deben capturar datos acerca del proceso de producción y utilizarlos para controlar las máquinas que manejan dentro de unos plazos de tiempo limitado y con frecuencia muy estricta.

Si una acción debe ejecutarse en cierto momento o intervalo de tiempo obligatoriamente, tendremos un sistema de tiempo real “riguroso”. Si puede ser aceptable que, de vez en cuando, no se cumpla un plazo, tendremos un sistema de tiempo real “no riguroso”. Los sistemas de audio digital o multimedia pertenecen a esta categoría. VxWorks y QNX son Sistemas Operativos de tiempo real muy conocidos.

3.4.6 Sistemas Operativos integrados

Considerando sistemas cada vez más pequeños, encontramos los sistemas integrados, que controlan pequeños dispositivos que no se consideran ni ordenadores, como por ejemplo: hornos, televisiones, coches, reproductores MP3, etc.

Suelen tener características propias de los sistemas de tiempo real, pero con limitaciones de tamaño, memoria y consumo de energía que los hace especiales.

3.4.7 Sistemas Operativos Móviles

Se verán más en profundidad más adelante, pero son los encargados de controlar un dispositivo móvil (Smartphone, Tablet, PDA, etc). Los Sistemas Operativos Móviles son más simples que los Sistemas Operativos tradicionales y están muy orientados al uso manual, la conectividad inalámbrica, los formatos multimedia y la interfaz de usuario.

3.4.8 Sistemas Operativos de tarjeta inteligente

Los Sistemas Operativos más pequeños se ejecutan en tarjetas inteligentes, que son dispositivos del tamaño de una tarjeta de crédito, que contienen un chip de CPU. Algunos de ellos solamente pueden desempeñar una función y otros realizan varias funciones en la misma tarjeta.

Algunas tarjetas inteligentes están orientadas hacia Java. Esto implica que la ROM de la tarjeta contiene un intérprete de la Máquina Virtual de Java (JVM Java Virtual Machine). Los applets (pequeños programas) de Java se descargan a la tarjeta y el intérprete de la JVM los procesa. Algunas tarjetas pueden manejar varios applets al mismo tiempo, dando pie a la multiprogramación y a la necesidad de planificación de tareas.

Debido a sus limitaciones en cuanto a velocidad, capacidad y proceso, suelen ser Sistemas Operativos muy primitivos.

3.5 Conclusión

En este tercer apartado se ha tratado la clasificación de los distintos Sistemas Operativos: por su estructura, por su capacidad, por la forma de ofrecer sus servicios y por el entorno Hardware en el que se ejecutan.

4 Evolución y tendencias de los Sistemas Operativos

4.1 Introducción

Después de conocer en profundidad el concepto más clásico de un Sistema Operativo y para qué sirve, es importante conocer cómo ha evolucionado a lo largo del tiempo y qué tendencias existen en la actualidad.

4.2 Generaciones de los Sistemas Operativos

Al estar íntimamente ligados con la arquitectura de los ordenadores en los que se ejecutaban, los Sistemas Operativos han ido evolucionando con el paso de los años. Por ello, es posible un acercamiento histórico a su evolución a través de las diversas generaciones de máquinas:

4.2.1 Primera generación (1945-1955): tubos de vacío y tableros enchufables.

Son los primeros sistemas informáticos, en los que los trabajos se lanzan de forma manual sucesivamente. El programador/operador interactúa directamente con el hardware, desde una consola. Los programas se crean en código máquina absoluto y el control de las funciones básicas se realiza mediante paneles enchufables. Se reservan tiempos de utilización para cada usuario, durante los cuales son de su exclusivo uso. A partir de 1950 aparecen las tarjetas perforadas que sustituyen la función de los tableros enchufables.

4.2.2 Segunda generación (1955-1965): transistores y sistemas de lotes.

Posteriormente se crean compiladores, ensambladores, bibliotecas de funciones comunes de E/S enlazables con las aplicaciones. Con el tiempo aparecen las unidades de cinta magnética y las impresoras de líneas. La planificación de trabajos (jobs) es manual (lista de reserva). El tiempo de preparación es largo pues se debe cargar el compilador en memoria, el programa fuente, obtener el programa compilado y enlazar las librerías de funciones comunes con el programa objeto. El sistema se organiza formando colas de trabajo (el operador/programador ya no tiene acceso directo a la máquina), gestionadas por un monitor residente siempre en memoria, que se encarga de cargar cada trabajo de la cola y pasarle el control de la UCP para que se ejecute. Una vez ejecutado el programa, éste devuelve el control de la UCP al monitor, que se encarga de cargar el siguiente programa en la cola. El objetivo es minimizar el tiempo de preparación de los trabajos y optimizar así el uso de la UCP. El uso de un monitor residente se apoya en la protección de memoria, la temporización y la ejecución de instrucciones privilegiadas. La preparación de los trabajos se realiza a través de un lenguaje de control de trabajos (JCL).

4.2.3 Tercera generación (1965-1980): circuitos integrados (CI) y multiprogramación.

Aparecen los circuitos integrados, que aportan una mejor relación precio/rendimiento sobre las máquinas anteriores. La UCP todavía pasa mucho tiempo inactiva debido a las esperas cuando se ejecutan operaciones de E/S. Para evitarlo se cargan varios programas en memoria simultáneamente y se alterna su ejecución. Esto se denomina multiprogramación, y se apoya en el uso de interrupciones de E/S y de controladores DMA. Para poder desarrollar sistemas multiprogramados es necesaria la gestión de la memoria y la planificación de trabajos (selección del siguiente programa para ejecutar). Como evolución aparecen los sistemas de tiempo compartido donde el tiempo de procesador se comparte entre programas de varios usuarios, pudiendo ser programas interactivos (como procesos de transacciones). A diferencia de la multiprogramación por lotes, donde se pretende maximizar la utilización de la UCP y se utiliza un lenguaje de control de trabajos, el tiempo compartido pretende minimizar el tiempo de respuesta, introduciéndose las órdenes a través de un terminal.

4.2.4 Cuarta generación (1980-hasta hoy): ordenadores personales.

Aparecen los circuitos LSI (Integración de gran escala), las máquinas se hacen más pequeñas, aumenta la potencia de cálculo y se abaratan los precios hasta permitir que un departamento de una empresa o una universidad tengan su propio ordenador. Los Sistemas Operativos evolucionan hacia sistemas interactivos cuyo objetivo es ser amable con el usuario. Se ponen en marcha las redes de ordenadores personales que ejecutan Sistemas Operativos en red y Sistemas Operativos distribuidos.

En un sistema operativo en red, los usuarios tienen conocimiento de la existencia de múltiples ordenadores y pueden acceder a máquinas remotas, así como copiar archivos de una máquina a otra. Cada máquina ejecuta su sistema operativo local y tiene un usuario propio. Un sistema operativo distribuido se presenta ante sus usuarios como un sistema uniprocador tradicional, los usuarios no conocen dónde se están ejecutando sus programas o dónde están ubicados sus archivos, toda la complejidad se maneja de forma automática y eficiente por medio del sistema operativo.

Los primeros Sistemas Operativos de microordenadores personales obligan a introducir las instrucciones mediante el teclado. Las investigaciones de Doug Engelbart en el Stanford Research Institute, en los años sesenta, le conducen a la invención de la interfaz gráfica de usuario GUI (Graphical User Interface), provista de ventanas, iconos, menús y ratón. Estas ideas fueron adoptadas por los investigadores de Xerox, incorporándolas a sus máquinas.

Steve Jobs, uno de los dos jóvenes que inventaron el ordenador Apple, vio una GUI en una visita al centro de investigación de Xerox, captando el valor potencial de esa tecnología. Se dedicó entonces a construir una Apple provista de GUI, que presentaba una interfaz amigable para el usuario, lo que significaba que iba dirigida a usuarios que no sólo carecían de conocimientos de ordenadores, sino que no tenían la menor intención de aprender.

Desde entonces, la evolución de los diferentes Sistemas Operativos de ordenadores personales, ha ido ligada a la existencia en ellos de una interfaz de usuario basada en GUI, que ha mantenido inalterables sus elementos básicos de ventanas, iconos, menús, ratón y ha comenzado a incorporar otros nuevos, como el reconocimiento de voz, los lápices digitales, los guantes interactivos, etc.

4.3 Tendencias de los Sistemas Operativos en la actualidad

Como ya decíamos anteriormente, la evolución de los Sistemas Operativos siempre ha estado muy ligada a la evolución de los equipos donde se ejecutan.

Por ello, en los últimos años ha habido dos grandes tendencias en el mundo de los Sistemas Operativos:

- Por un lado, se ha pasado de los sistemas de 32bits a los de 64 bits.
- Por otro lado, el auge de los dispositivos móviles ha provocado un auge de nuevos Sistemas Operativos Móviles.

Vamos a tratarlo más en detalle.

4.3.1 Sistemas de 32bits y de 64 bits

Al hablar de 32 y de 64 bits se hace referencia a la arquitectura del procesador del ordenador.

El procesador es el encargado de realizar los cálculos necesarios para ejecutar una operación solicitada por el usuario. Para ello, el procesador maneja paquetes de información a una velocidad concreta. La diferencia de las dos plataformas radica en que el procesador de 64 bits gestiona el doble de información en el mismo ciclo de reloj que el de 32 bits, lo que permite acceder a mayor cantidad de memoria y procesar bloques de datos más grandes, aumentando el rendimiento y la velocidad de proceso.

Dado que el Sistema Operativo está íntimamente ligado al Hardware, se han desarrollado Sistemas Operativos para ambas arquitecturas, con el fin de poder aprovechar las múltiples ventajas de la arquitectura de 64 bits.

4.3.2 Sistemas Operativos Móviles

Los Sistemas Operativos móviles son los encargados de controlar un dispositivo móvil (Smartphone, Tablet, PDA, etc). Mientras que el típico portátil puede considerarse un dispositivo móvil, el sistema operativo que utiliza no se considera que forme parte de los Sistemas Operativos Móviles ya que han sido diseñados originalmente para ordenadores personales que históricamente no necesitaban características móviles. Esta distinción cada vez se está haciendo más difusa por la llegada de nuevos Sistemas Operativos híbridos creados para ambos paradigmas, como es el caso de Windows 8.

Los Sistemas Operativos Móviles son más simples que los Sistemas Operativos tradicionales y están muy orientados al uso manual, la conectividad inalámbrica, los formatos multimedia y la interfaz de usuario.

El auge de los dispositivos móviles de los últimos años ha provocado consecuentemente un aumento de alternativas de Sistemas Operativos Móviles. Si bien en un principio cada fabricante desarrolló su propio Sistema Operativo con su plataforma para su dispositivo, en 2008, Google liberó la primera versión de Android, Sistema Operativo que funcionaba en múltiples dispositivos.

Ello ha provocado un gran cambio en el mundo de los Sistemas Operativos móviles de un tiempo a esta parte. No hay más que ver el siguiente cuadro de Gartner sobre estimaciones de ventas de Sistemas Operativos Móviles por cuota de mercado:

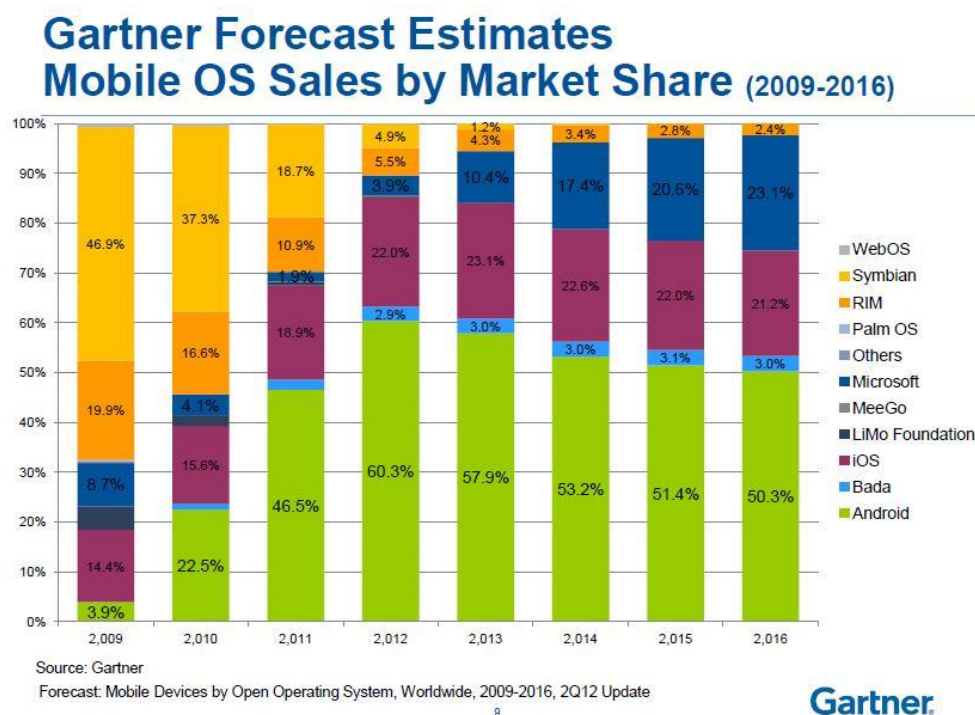


Figura 15: Estimaciones de ventas de Sistemas Operativos Móviles por cuota de mercado

En el gráfico se puede apreciar cómo Sistemas Operativos como Symbian que suponía un 50% de la cuota de mercado en 2009, actualmente han desaparecido y su espacio ha sido ocupado claramente por Android.

A continuación vamos a tratar los diferentes Sistemas Operativos Móviles más conocidos existentes en la actualidad:



Figura 16: Sistemas Operativos Móviles

- Android

Android es un Sistema Operativo Móvil basado en el kernel de Linux y desarrollado inicialmente por la empresa Android Inc. En 2005 Google adquirió la empresa, lo que dio pie a multitud de rumores de que Google planeaba entrar en el mercado de los teléfonos móviles. Sin embargo, Google se dedicó al desarrollo de una plataforma para dispositivos móviles basada en Linux que presentó a fabricantes de dispositivos y operadoras de comunicaciones con la promesa de proporcionar un sistema flexible y actualizable.

En 2007 se presentó la fundación *Open Handset Alliance (OHA)*, un consorcio de 78 compañías de hardware, software y telecomunicaciones dedicado a avanzar en los estándares abiertos de los dispositivos móviles. Y junto con la OHA, Google presentó Android, una plataforma libre para dispositivos móviles basada en la versión 2.6 de Linux.

Android es el Sistema Operativo Móvil más instalado a nivel mundial en smartphones. Casi todo Android es software libre, pero una gran cantidad del software que llevan los dispositivos Android, como el Google Play, la búsqueda de Google o la música, es propietario.

El Sistema Operativo Android se comenzó a utilizar exclusivamente en móviles, siendo el HTC Dream el primer móvil con sistema Android. Después de la versión 2.0 se comenzó a utilizar también en tabletas. Las versiones de Android llevan el nombre de dulces o postres. La versión actual de Android es la 5, llamada Lollipop.

Android se compone de aplicaciones que se ejecutan en un framework Java sobre el núcleo de bibliotecas de Java en una máquina virtual con compilación en tiempo de ejecución. Las bibliotecas escritas en C incluyen, entre otros, un administrador de interfaz gráfica, un framework, una base de datos relacional, un interfaz de programación de API gráfica, un motor de renderizado, un motor gráfico, etc.

Android ha sido criticado en multitud de ocasiones por la reiterada aparición de nuevas versiones que no funcionan correctamente en el hardware diseñado para versiones anteriores. Google intenta acabar con este problema con su plataforma de servicios *Google Play*, separando al máximo las aplicaciones del sistema, de forma que se puedan actualizar de forma independiente.

Por último, Android es considerado uno de los modelos de negocio más exitosos. Al contrario que otros sistemas operativos propietarios, Android se desarrolla de forma abierta y se puede acceder tanto al código fuente como a la lista de incidencias donde se pueden ver problemas aún no resueltos y reportar problemas nuevos.

- iOS

iOS es el Sistema Operativo Móvil de los dispositivos de Apple. Es el segundo Sistema Operativo Móvil más instalado a nivel mundial en smartphones, pero el que más beneficios genera debido a la

gran competencia existente entre los fabricantes basados en Android. Es un sistema propietario que se desarrolló originalmente para el iPhone, aunque actualmente se utiliza en los dispositivos iPhone, iPod Touch, iPad y la segunda generación de Apple TV.

Como hemos visto en la gráfica anterior, la cuota de mercado del iOS se sitúa en torno al 20%.

iOS realmente es un derivado del Sistema Operativo OS X, por lo que es un sistema operativo de tipo Unix. Cuenta con cuatro capas de abstracción: la capa del núcleo del sistema operativo, la capa de *Servicios Principales*, la capa de *Medios* y la capa de *Cocoa Touch*. Los elementos de control del Sistema Operativo consisten en deslizadores, interruptores y botones.

Apple reveló la existencia del iPhone OS (predecesor de iOS) en el año 2007 y su enorme éxito se debió al anuncio por parte de Steve Jobs de un Kit de desarrollo de software (SDK) que estaría disponible para terceros y desarrolladores, con el fin de que pudieran crear aplicaciones para sus dispositivos.

iOS no permite Adobe Flash ni Java, por considerar que Flash es inseguro, contiene errores, consume mucha batería, es incompatible con interfaces multitouch e interfiere con el servicio App Store.

- **Windows Phone**

Windows Phone es un Sistema Operativo Móvil propietario desarrollado por Microsoft para el mercado de consumo. Es el tercer Sistema Operativo más utilizado en dispositivos móviles a nivel mundial, por detrás de Android y iOS.

Fue presentado por Microsoft en el año 2010 y cuenta con una nueva interfaz de usuario totalmente renovada orientada a sus servicios OneDrive, Skype y Xbox Live y que también integra servicios de Facebook y Google. Los dispositivos que integran este Sistema Operativo mayoritariamente son Nokia, HTC y Samsung.

En su filosofía de los últimos años iniciada con Windows 8, Microsoft anunció en enero de 2015 que dará de baja Windows Phone, para centrarse en el desarrollo de Windows 10, un único sistema operativo más versátil, disponible para todo tipo de plataformas: ordenadores personales, smartphones y tabletas.

Una de las principales características de la nueva interfaz de usuario integrada en Windows Phone, Windows 8 y posteriores es que la pantalla de inicio se compone de *Live Tiles*, mosaicos dinámicos que son enlaces a aplicaciones u objetos individuales. Estos enlaces actualizan frecuentemente su información, informando directamente al usuario de cualquier cambio sin tener que entrar en él.

- **BlackBerry**

BlackBerry desarrolló inicialmente Blackberry OS, un Sistema Operativo Móvil propietario para sus teléfonos inteligentes.

En mayo de 2013, se lanzó la versión BlackBerry 10 del Sistema Operativo que fue desarrollada completamente desde cero, está basada en QNX y cuenta con numerosas mejoras solicitadas por los usuarios.

Hasta entonces, BlackBerry había sido uno de los mayores fabricantes de móviles del mundo, muy utilizado en el mundo empresarial y muy famosa por su teclado QWERTY incorporado y por su conocido servicio de correo en el móvil integrado desde 1999. Pero con la llegada de los Smartphones Blackberry no supo posicionarse a tiempo y perdió cuota de mercado hasta el 0,4% en 2014.

El Sistema Operativo BlackBerry 10 sigue estando muy dirigido al entorno empresarial y utiliza una combinación de gestos y toques para la navegación y el control con las que es posible introducir comandos en el dispositivo sin tener que pulsar ningún botón físico. Además incorpora el *Blackberry*

Balance, la división de forma nativa entre un entorno personal y un entorno profesional, muy demandados por los usuarios actualmente. Los entornos están totalmente separados, hasta el punto de que no se pueden pasar datos, ni fotos o similares de un entorno a otro. Y para pasar de uno a otro se debe utilizar una clave. Por último, incorpora el *BlackBerry Hub*, que es un centro de notificaciones que integra todas las cuentas de correo y de redes sociales del usuario en una única aplicación.

- **Firefox OS**

Firefox OS, cuyo nombre de proyecto es Boot to Gecko, más conocido como B2G, es un Sistema Operativo Móvil abierto, basado en el kernel de Linux y desarrollado por Mozilla.

Mozilla indica que creó B2G por un deseo de demostrar que la Web basada en estándares abiertos tiene suficiente potencial para ser una alternativa competitiva a los numerosos desarrollos de aplicaciones de un único proveedor que ofrecen los Sistemas Operativos Móviles dominantes.

Firefox OS está diseñado para proporcionar un sistema alternativo, completo y basado en una comunidad para dispositivos móviles, utilizando estándares abiertos como HTML5, JavaScript, un modelo fuerte de permisos, APIs web abiertas para comunicarse directamente con el hardware del teléfono móvil y un mercado de aplicaciones.

Firefox OS fue lanzado en febrero de 2012 y los fabricantes que se han comprometido a crear dispositivos para él son LG, ZTE y Huawei.

El Sistema Operativo está compuesto por tres capas de software principales:

- Gonk es una combinación entre el kernel de Linux y la capa de abstracción del hardware que usa Android.
- Gecko es el motor de renderizado y la capa de aplicación de servicios en tiempo de ejecución. Implementa estándares abiertos para HTML, CSS y JavaScript.
- Gaia es la interfaz de usuario de Firefox OS y controla todo lo que se muestra en la pantalla.

4.4 Conclusión

Una vez introducido en profundidad el concepto de Sistema Operativo y sus elementos y características básicos, en este último apartado se ha tratado la evolución de los Sistemas Operativos a lo largo del tiempo y las tendencias existentes en la actualidad.

Para ello, se han introducido las cuatro generaciones que han existido de Sistemas Operativos desde sus inicios, muy ligadas a los equipos existentes en cada época.

Y por último para hablar de las tendencias de los Sistemas Operativos en la actualidad se ha tratado:

- Por un lado de uno de los cambios más sonados de los últimos años en cuanto a Sistemas Operativos: Sistemas de 32 o de 64 bits.
- Y, por otro, de los Sistemas Operativos Móviles con más éxito en la actualidad: Android, iOS, Windows Phone, Blackberry OS y Firefox OS.

RESUMEN ESQUEMÁTICO

1 Introducción

El Sistema Operativo es el software más importante de un ordenador, ya que es el que controla todos los recursos físicos del ordenador y el encargado de ofrecer las bases y servicios sobre los que se ejecutarán el resto de aplicaciones software. El Sistema Operativo es el software más cercano al propio hardware del ordenador y sobre él se desarrollan las aplicaciones, cuyo objetivo principal es el de resolver problemas de los usuarios.

2 Componentes del Sistema Operativo

Un Sistema Operativo tiene cuatro cometidos principales:

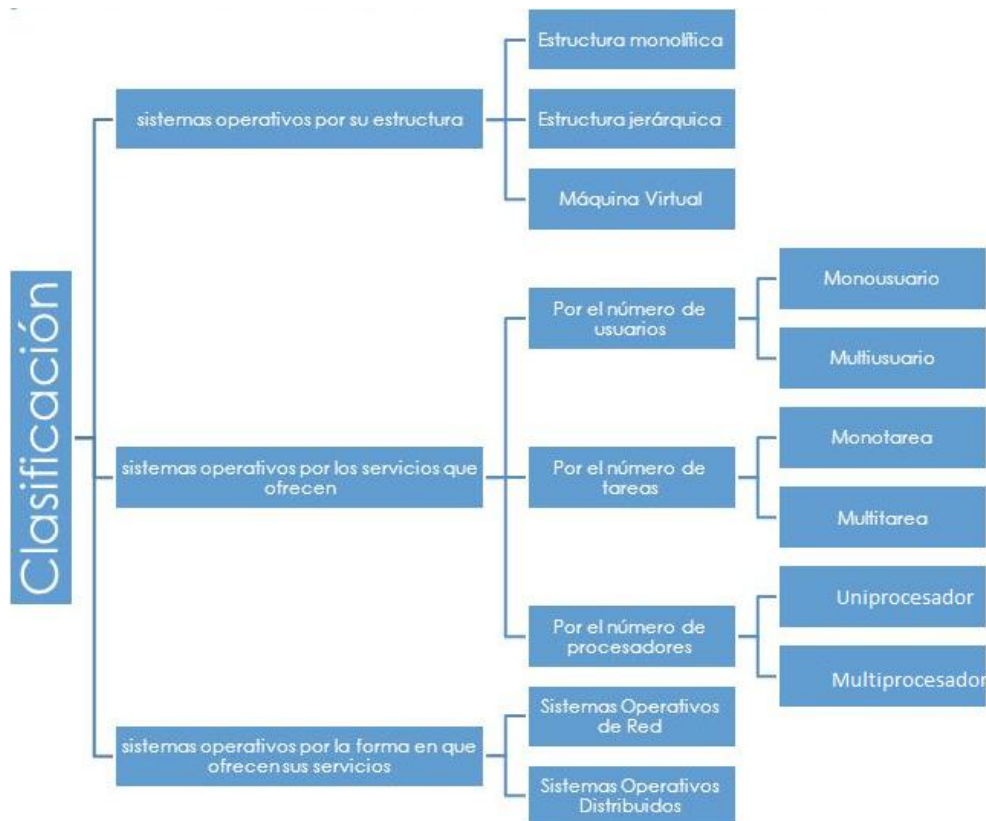
- La gestión de procesos.
- La gestión de memoria.
- El sistema de archivos.
- La gestión de la Entrada/Salida.

Los componentes encargados de llevar a cabo estos cometidos dentro del Sistema Operativo son:

- **Administrador de procesos.**
- **Administrador de memoria principal.**
- **Administrador del sistema de E/S.**
- **Administrador de archivos.**
- **Sistema de protección.**
- **Sistema de comunicación.**

3 Clasificación de los Sistemas Operativos

En esta sección se describirán las características que clasifican a los Sistemas Operativos, mostrando varias clasificaciones básicas



4 Evolución y Tendencias de los Sistemas Operativos

Después de conocer en profundidad el concepto más clásico de un Sistema Operativo y para qué sirve, es importante conocer cómo ha evolucionado a lo largo del tiempo y qué tendencias existen en la actualidad.

4.1 Generaciones de los Sistemas Operativos

- Primera Generación (1945-1955)
- Segunda Generación (1955-1965)
- Tercera Generación (1965-1980)
- Cuarta Generación

4.2 Tendencias de los Sistemas Operativos en la Actualidad

- Sistemas de 32 y 64 bits
- Sistemas Operativos Móviles
 - Android
 - iOS
 - Windows Phone
 - BlackBerry
 - Firefox OS

PREGUNTAS DE TEST

- 1. Los programas que constituyen el núcleo del Sistema Operativo siempre se encuentran almacenados en:**
 - a) Disco
 - b) Memoria principal
 - c) Memoria masiva
 - d) Biblioteca de programas

- 2. ¿Cuál de los siguientes términos es el menos relacionado con los Sistemas Operativos?**
 - a) Sincronización
 - b) Organigrama
 - c) Concurrencia
 - d) Semáforos

- 3. ¿Qué es lo que un ordenador con una sólo CPU no podría realizar?**
 - a) Atender a varios usuarios
 - b) Multiprocesamiento
 - c) Acceder a Internet
 - d) Multitarea

- 4. ¿Cuáles de las siguientes tareas no es función del kernel de UNIX?**
 - a) Controlar recursos del sistema
 - b) Controlar los periféricos
 - c) Compartición de recursos
 - d) Controlar la conexión a internet

- 5. ¿Cuál de las siguientes transiciones entre estados de un proceso es incorrecta?**
 - a) En ejecución -> Bloqueado
 - b) En ejecución -> Listo
 - c) Listo -> Bloqueado
 - d) Bloqueado -> Listo

- 6. La gestión de la memoria no requiere**
 - a) Protección
 - b) Filtrado
 - c) Compartición
 - d) Reubicación

- 7. ¿Cuántas generaciones de Sistemas Operativos existen?**
 - a) 2
 - b) 4
 - c) 7
 - d) 10

- 8. ¿Cuándo liberó Google la primera versión de Android?**
 - a) 2000
 - b) 2005
 - c) 2008
 - d) 2010

- 9. ¿Cuál es el Sistema Operativo más utilizado a nivel mundial?**
 - a) Android
 - b) iOS
 - c) Windows Phone
 - d) Cyanogen

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	b
2	b
3	b
4	d
5	C
6	B
7	b
8	c
9	a

BIBLIOGRAFÍA BÁSICA

SILBERSCHATZ, A., GALVIN, P.B. y GAGNE, G.: *Operating Systems Concepts (Essentials)*, 9th Edition, Wiley, 2012.

STALLINGS, William: *Operating Systems: Internals and Design Principles*, 8th Edition, Pearson, 2015.

TANENBAUM, Andrew S., HERBERT, Bos: *Modern Operating Systems*, 4th Edition. Pearson, 2015.

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

DOEPPNER, T. W.: *Operating Systems in Depth*. Wiley, 2011.

HOLCOMBE, Charles, HOLCOMBE, Jane: *Survey of Operating Systems (Mike Meyers' Computer Skills)*, 2nd Edition, 2005.

MCHOES, Ann Mciver, FLYNN, Ida M.: *Understanding Operating Systems*, 7th Edition. Cengage Learning, 2014.

PALMER, Michael, WALTERS, Michael: *Guide to Operating Systems*, 4th Edition. Thomson/Course Technology. 2011.

SILBERSCHATZ, A., GALVIN, P.B. y GAGNE, G.: *Operating Systems Concepts with Java*, Wiley, 2009.

TANENBAUM, Andrew S., WOODHULL, Albert S.: *Operating Systems Design and Implementation*, 3rd Edition. Pearson, 2006.